YAPAY ZEKÂYA GİRİŞ · TÜRKÇE DERS NOTU

06

Kısıt Tatmin Problemleri

Constraint Satisfaction Problems (CSP)

Eksiksiz bölüm anlatımı, Türkçe çeviri, algoritmalar, şekiller, kısıt grafikleri ve adım adım çözümlü örneklerle hazırlanmış çalışma kaynağı.

Kaynak: Stuart Russell & Peter Norvig, *Artificial Intelligence: A Modern Approach*, 3. Baskı, Bölüm 6 — kullanıcı tarafından paylaşılan 6.1–6.5 metinleri.

İçerik: CSP tanımları · Kısıt yayılımı · Geri izleme · Yerel arama · Problem yapısı

Dil: Türkçe | **Düzey:** Lisans / başlangıç | **Hazırlanma:** Ekim 2026

Bu ders notu nasıl kullanılmalı?

Bu belge, paylaştığınız İngilizce kitap metninin **6.1–6.5 ana bölümlerini** aynı sıra ile Türkçeye aktarır; metni izleyen “Açıklama”, “Adım adım örnek” ve “Dikkat” kısımları öğrenmeyi kolaylaştırmak üzere eklenmiştir. Kitabın şekilleri, kullanıcı tarafından gönderilen görsellerden alınmıştır. Oluşturulan ek grafikler ve çözümlü örnekler ayrıca belirtilmiştir.

Konular ilk kez öğreniliyorsa önce **değişken–değer kümesi–kısıt** üçlüsü, sonra **tutarlılık**, ardından **arama** ve son olarak **problemin yapısından yararlanma** fikirleri takip edilmelidir. Her algoritmanın amacı ile tamlık/başarısızlık davranışı birbirinden ayrılmalıdır.

İçindekiler

6.1 Kısıt tatmin problemlerinin tanımlanması 3
6.1.1 Harita boyama 4
6.1.2 Fabrika işi çizelgeleme 5
6.1.3 CSP biçiminin çeşitleri 6
6.2 Kısıt yayılımı ve çıkarım 10
6.2.1 Düğüm tutarlılığı 10
6.2.2 Yay tutarlılığı ve AC-3 10
6.2.3 Yol tutarlılığı 11
6.2.4 k-tutarlılığı 12
6.2.5 Genel kısıtlar ve sınır yayılımı 12
6.2.6 Sudoku örneği 13
6.3 CSP için geri izlemeli arama 15
6.3.1 Değişken ve değer sıralaması 16
6.3.2 Arama ile çıkarımı birlikte yürütme 17
6.3.3 Akıllı geri izleme ve kısıt öğrenme 19
6.4 CSP için yerel arama 21
6.5 Problemlerin yapısı 24
Karşılaştırmalar, alıştırmalar ve cevaplar 29

6.1 Kısıt Tatmin Problemlerinin Tanımlanması

3. ve 4. bölümlerde problemlerin bir durum uzayında arama yapılarak çözülebileceğini gördük. Durumlar, probleme özgü sezgisel yöntemlerle değerlendirilebilir veya hedef olup olmadıkları bakımından test edilebilir. Bununla birlikte klasik arama algoritması açısından her durum, iç yapısı dikkate alınmayan bölünemez bir **kara kutu** gibidir.

Bu bölümde, çok farklı problemleri daha verimli çözmek için her durumu değişkenlerin ve bu değişkenlere atanmış değerlerin oluşturduğu **parçalı gösterim (factored representation)** ile ifade ederiz. Bir problem ancak bütün değişkenler, üzerlerindeki kısıtları sağlayan değerlere sahip olduğunda çözülür. Bu şekilde tanımlanan problem **kısıt tatmin problemi (Constraint Satisfaction Problem, CSP)** adını alır.

CSP algoritmalarının temel avantajı, durumların iç yapısından yararlanmaları ve probleme özel bilgi gerektirmeyen genel amaçlı sezgisel yöntemler kullanabilmeleridir. Kısıtlarla çelişen değişken/değer kombinasyonlarını erken belirleyerek arama uzayının büyük kısımlarını tek hamlede eleyebilirler.

$$\mathbf{CSP = (X, D, C)}$$

X — değişkenler: $X = \{X_1, X_2, \ldots, X_n\}$. Her X_i problemde bir karar unsurudur.

D — değer kümeleri (domains): $D = \{D_1, D_2, \ldots, D_n\}$. $D_i = \{v_1, \ldots, v_k\}$, X_i değişkeninin alabileceği değerlerdir.

C — kısıtlar: Hangi değişkenlerin hangi değer kombinasyonlarını birlikte alabileceğini tanımlar. Bir kısıt, kapsam (scope) ile izin verilen ilişki (relation) çiftidir: C_i = (scope, rel). İlişki açıkça izin verilen kombinasyonlar listesi olarak ya da bu kombinasyonları sınayan soyut bir koşul olarak gösterilebilir.

Örneğin X_1 ve X_2 değişkenlerinin her ikisi de {A, B} değer kümesine sahipse, **farklı değer alma** kısıtı ya $((X_1, X_2), \{(A,B), (B,A)\})$ biçiminde listelenir ya da kısa şekilde $X_1 \neq X_2$ olarak ifade edilir.

Atama, tutarlılık ve çözüm

Bir CSP'nin durumu, değişkenlerin bir bölümüne veya tamamına değer atanmasıyla tanımlanır: $\{X_i = v_i, X_j = v_j, \ldots\}$. Hiçbir kısıtı ihlal etmeyen atama **tutarlı / geçerli (consistent/legal)**, bütün değişkenlere değer verilmiş atama **tam (complete)**, yalnızca bir kısmına değer verilmiş atama ise **kısmi (partial)** olarak adlandırılır. **Çözüm, aynı anda hem tam hem tutarlı atamadır.**

Atama türü	Örnek	Çözüm mü?
Kısmi ve tutarlı	A = 1, B atanmamış	Hayır; eksik
Tam ve tutarsız	A = 1, B = 1; A ≠ B kısıtı var	Hayır; ihlal var
Tam ve tutarlı	A = 1, B = 2; A ≠ B sağlanıyor	**Evet**

Öğrenci örneği — Üç sunumu planla. Ali, Ayşe ve Mehmet'in sunum saatleri 09.00, 10.00 ve 11.00 olabilir; aynı anda iki sunum yapılamaz. X = {Ali, Ayşe, Mehmet}; her değer kümesi {9, 10, 11}; kısıt Alldiff(Ali, Ayşe, Mehmet) olsun. {Ali=9, Ayşe=10, Mehmet=11} çözümdür. {Ali=9, Ayşe=9, Mehmet=11} ise tam olmasına rağmen geçersizdir.

6.1.1 Örnek problem: Harita boyama (Map Coloring)

Avustralya'nın eyaletlerini ve bölgelerini gösteren haritayı düşünelim (Şekil 6.1(a)). Her bölge kırmızı, yeşil veya mavi boyanacaktır; ancak komşu bölgeler aynı renk olamaz. Bu bir CSP olarak şu şekilde ifade edilir:

X = {WA, NT, Q, NSW, V, SA, T} D_i = {kırmızı, yeşil, mavi}

WA = Batı Avustralya; NT = Kuzey Bölgesi; Q = Queensland; NSW = Yeni Güney Galler; V = Victoria; SA = Güney Avustralya; T = Tazmanya.

Dokuz komşuluk ilişkisi bulunduğundan dokuz kısıt vardır: SA ≠ WA; SA ≠ NT; SA ≠ Q; SA ≠ NSW; SA ≠ V; WA ≠ NT; NT ≠ Q; Q ≠ NSW; NSW ≠ V. Örneğin SA ≠ WA kısıtı, iki bölge için (kırmızı, yeşil), (kırmızı, mavi), (yeşil, kırmızı), (yeşil, mavi), (mavi, kırmızı), (mavi, yeşil) çiftlerine izin verir.

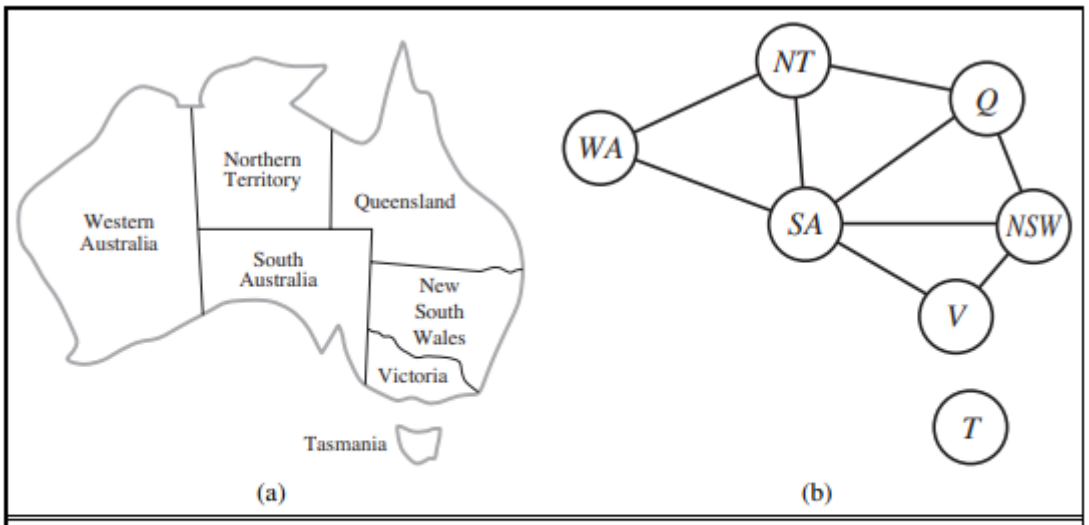
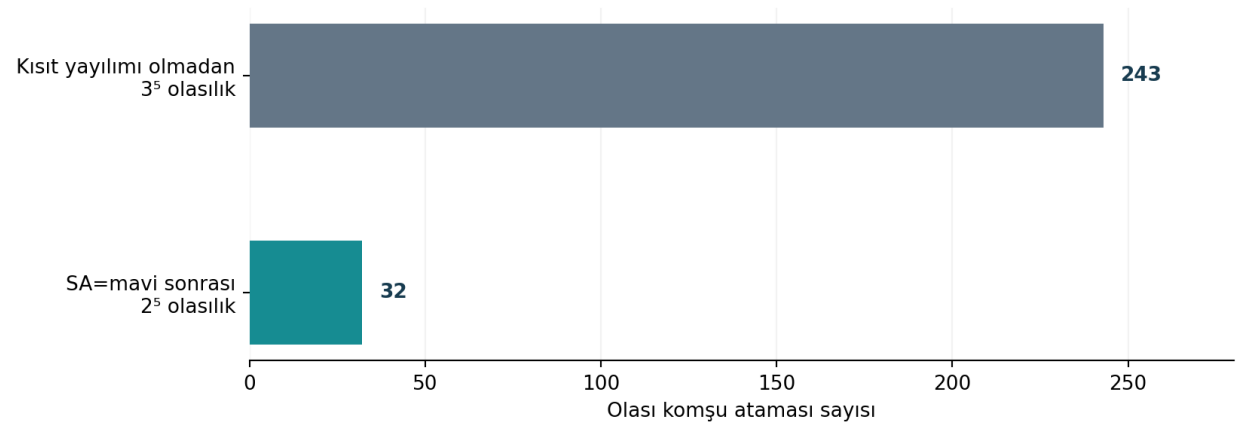


Figure 6.1 (a) The principal states and territories of Australia. Coloring this map can be viewed as a constraint satisfaction problem (CSP). The goal is to assign colors to each region so that no neighboring regions have the same color. (b) The map-coloring problem represented as a constraint graph.

Şekil 6.1. (a) Avustralya'nın başlıca eyalet ve bölgeleri. Komşuların farklı renge boyanması bir CSP'dir. (b) Harita boyama problemine karşılık gelen kısıt grafiği: daireler değişkenleri, çizgiler komşuluk kısıtlarını gösterir.

Örneğin şu atama bütün kısıtları sağlar: WA=kırmızı, NT=yeşil, Q=kırmızı, NSW=yeşil, V=kırmızı, SA=mavi ve T=kırmızı. Kısıt grafiğinde iki değişken arasında bağlantı bulunması, o iki değişkeni içeren bir ikili kısıt olduğu anlamına gelir. Tazmanya'nın diğer bölgelere bağlantısı yoktur.

Neden CSP kullanalım? Problemi doğal biçimde gösterir; hazır bir CSP çözücüsü kullanma olanağı sunar; ayrıca kısıtlar arama uzayını hızla küçültebilir. SA=mavi seçildiğinde beş komşusunun hiçbiri mavi olamaz. Kısıtlardan yararlanmayan kaba bir karşılaştırmada beş değişken için $3^5=243$ renk kombinasyonu bulunurken, bu seçimden sonra $2^5=32$ kalır: yaklaşık %86,8 azalma. Diğer komşuluk kısıtları ayrıca elenebilir.



Ek Grafik A. SA=mavi çıkarımı sonrasında beş komşunun kaba kombinasyon sayısı; grafiğin sayıları tek bir çıkarımın etkisini göstermektedir.

Klasik aramada durumun hedef olup olmadığı sorulur; CSP'de ise bir kısmi atamanın neden başarısız olacağı da anlaşılabilir. Örneğin bir değişken için hiçbir geçerli değer kalmaması durumunda o atamadan türeyen tüm dallar elenir. Böylece geleneksel arama açısından çok zor problemler pratik biçimde çözülebilir.

> **Adım adım:** (1) SA=mavi ata. (2) WA, NT, Q, NSW ve V'nin değer kümelerinden maviyi sil. (3) WA=kırmızı, NT=yeşil seç. (4) NT=yeşil ve SA=mavi olduğundan Q=kırmızı kalır. (5) Q=kırmızı ve SA=mavi olduğundan NSW=yeşil olur. (6) NSW=yeşil ve SA=mavi olduğundan V=kırmızı seçilir. T bağımsız olduğundan istediğin rengi alır.

6.1.2 Örnek problem: Fabrika işi çizelgeleme (Job-Shop Scheduling)

Fabrikalar, gün boyunca gerçekleştirilecek işleri farklı kaynak, sıra ve zaman kısıtları altında planlamak zorundadır. Bu problemlerin önemli bir bölümü CSP teknikleriyle çözülebilir. Bir otomobil montajı örneğinde her görev bir değişkendir; değeri görevin başlama zamanını dakika cinsinden belirtir.

Montajın küçük bir parçasında 15 görev olsun: ön ve arka aksların takılması (2); dört tekerleğin takılması (4); her tekerleğin somunlarının sıkılması (4); jant kapaklarının takılması (4); son kontrol (1). Değişkenler AxleF, AxleB, WheelRF, WheelLF, WheelRB, WheelLB, NutsRF, NutsLF, NutsRB, NutsLB, CapRF, CapLF, CapRB, CapLB ve Inspect olarak adlandırılır. RF/LF sağ/sol ön, RB/LB sağ/sol arka konumlarını belirtir.

Öncelik kısıtları (Precedence Constraints)

Bir T_1 görevi T_2'den önce bitmeli ve T_1 görevinin süresi d_1 ise temel kısıt $T_1 + d_1 \leq T_2$ olur. Bir aksın montajı 10 dakika sürdüğünden şu kısıtlar uygulanır:

$$\mathbf{AxleF+10 \le WheelRF;\quad AxleF+10 \le WheelLF}$$
$$\mathbf{AxleB+10 \le WheelRB;\quad AxleB+10 \le WheelLB}$$

Her tekerlek takıldıktan sonra somunlar sıkılır ve en son kapak takılır. Bir tekerleğin montajı 1, somun sıkma 2, kapak montajı 1 dakika sürsün. Sağ ön tekerlek için WheelRF+1 ≤ NutsRF ve NutsRF+2 ≤ CapRF kısıtları yazılır. Aynı çift kısıt LF, RB ve LB için de geçerlidir.

> **Hesaplanmış örnek:** WheelRF=12 ise somun sıkma işlemi 12+1=13. dakikadan önce başlayamaz. NutsRF=13 ise kapak takma CapRF ≥ 13+2=15 olmalıdır. Kapak 15'te başlarsa 16'da biter. Eşitsizlikler görev sırasını somut bir matematiksel koşula dönüştürür.

Ayrışmalı kısıt (Disjunctive Constraint)

Dört işçi bulunsa da iki aks montajı aynı özel aleti kullanmak zorunda olsun. İki aks montajı üst üste gelemez; önce biri ya da diğeri yapılmalıdır:

$$\mathbf{(AxleF + 10 \le AxleB)\ VEYA\ (AxleB + 10 \le AxleF)}$$

Bu, aritmetik eşitsizliklerle mantıksal VEYA'nın birlikte kullanıldığı bir kısıttır; yine iki değişkenin izin verilen başlama zamanı çiftlerine karşılık gelir. Son kontrol bütün diğer görevlerden sonra başlamalıdır: her $X \neq$ Inspect görevi için $X+d_x \le$ Inspect. Inspect işlemi ayrıca 3 dakika sürer.

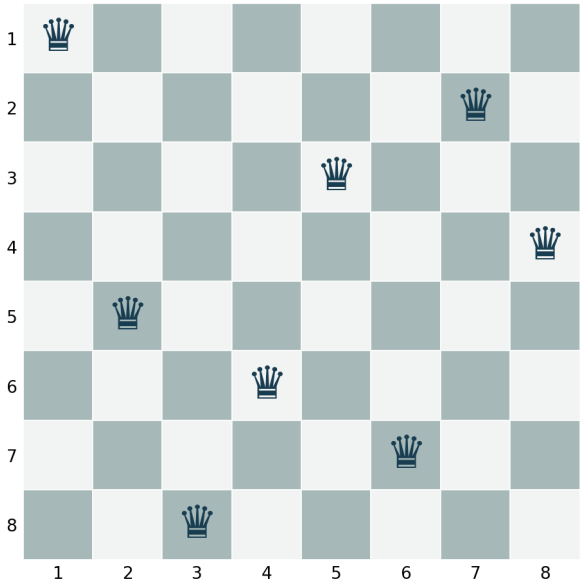
Bütün işlerin 30 dakika içerisinde bitirilmesi istenirse kitap başlama zamanlarının değer kümelerini $D_i=\{1,2,\ldots,27\}$ ile sınırlar. Bu seçim, son kontrolün üç dakika sürmesiyle ilişkilidir. Ancak bütün görevlerin zamanında tamamlanabilmesi için süre ve öncelik kısıtları da sağlanmalıdır; Inspect+3 ≤ 30 son kontrolün üst sınırını açıkça belirtir.

Bu küçük problem kolay olsa da gerçek iş atölyesi çizelgeleme CSP'lerinde binlerce değişken bulunabilir. CSP biçiminde ifade edilmesi güç olan karmaşık kısıtlar için daha gelişmiş planlama teknikleri de kullanılır.

6.1.3 CSP biçiminin çeşitleri (Variations on the CSP Formalism)

Ayrık, sonlu ve sonsuz değer kümeleri

En basit CSP türünde değişkenler **ayrık ve sonlu** değerler alır. Harita boyama ve süre sınırı olan çizelgeleme problemleri buna örnektir. Sekiz vezir probleminde de $Q_1,\ldots,Q_8$ değişkenleri, her sütundaki vezirin satırını belirtir; her biri $\{1,2,\ldots,8\}$ değer kümesine sahiptir. Vezirler aynı satırda veya çaprazda bulunmamalıdır: $Q_i \neq Q_j$ ve $|Q_i - Q_j| \neq |i-j|$.



Ek Grafik B. Sekiz vezir probleminin geçerli bir yerleşimi. Her sütunda bir vezir bulunur; aynı satır ve çaprazlar yasaktır.

Ayrık bir değer kümesi **sonsuz** da olabilir: bütün tam sayılar veya karakter dizileri gibi. Bitiş zamanı olmayan bir iş çizelgelemesinde başlangıç zamanları sonsuza kadar uzanabilir. Bu durumda izin verilen bütün çiftleri listelemek yerine $T_1+d_1 \leq T_2$ biçimindeki koşulları doğrudan işleyen bir kısıt dili gerekir.

Tam sayı değişkenlerde doğrusal kısıtlar için özel çözüm yöntemleri geliştirilmiştir; genel doğrusal olmayan tam sayı kısıtların tüm örnekleri için karar veren bir algoritma yoktur. Bu, her tekil doğrusal olmayan problemin çözülemeyeceği anlamına gelmez.

Sürekli değer kümeleri ve doğrusal programlama

Sürekli değerli CSP'ler yöneylem araştırmasında yaygındır. Örneğin Hubble Uzay Teleskobu gözlemlerinin çok hassas başlangıç/bitiş zamanları; astronomik koşullara, önceliklere ve güç sınırlarına uymalıdır. En bilinen kategori, kısıtların doğrusal eşitlik/eşitsizlikler olduğu **doğrusal programlama (linear programming)** problemleridir. Bu problemler değişken sayısına göre polinom zamanda çözülebilir. Kuadratik programlama ve ikinci dereceden konik programlama gibi başka modeller de vardır.

Değer kümesi	Örnek	Temsil
Sonlu ayrık	Harita rengi	{kırmızı, yeşil, mavi}
Sonsuz ayrık	Yukarı sınırsız gün sayısı	{1,2,3,...}
Sürekli	Robot kolunun açısı	[0,180] aralığındaki gerçek sayılar

Tekli, ikili, üçlü ve genel kısıtlar

Tekli (unary): Tek bir değişkeni sınırlar; örneğin SA ≠ yeşil. Düğümün değer kümesi {kırmızı,mavi} olarak daralır. **İkili (binary):** İki değişkeni ilişkilendirir; örneğin SA ≠ NSW. Yalnızca ikili kısıtları bulunan CSP bir kısıt grafiğiyle gösterilebilir. **Üçlü (ternary):** Örneğin Between(X,Y,Z), Y'nin X ile Z arasında bulunması koşuludur. **Genel (global):** İstenilen sayıda değişkene uygulanır; problemdeki bütün değişkenleri kapsaması zorunlu değildir.

En yaygın genel kısıt **Alldiff**'tir: kapsamındaki bütün değişkenler birbirinden farklı değerler almalıdır. Sudoku'da bir satır, sütun veya 3×3 kutudaki dokuz hücre için Alldiff uygulanır.

Şifreli aritmetik (Cryptarithmetic) ve kısıt hipergrafı

Bir şifreli aritmetik probleminde her harf farklı bir rakamı gösterir. Şekil 6.2(a)'daki TWO+TWO=FOUR örneğinde harfler F,T,U,W,R,O'dur. Başta sıfır bulunamaz: T ≠ 0 ve F ≠ 0. Ayrıca Alldiff(F,T,U,W,R,O) kısıtı bulunur.

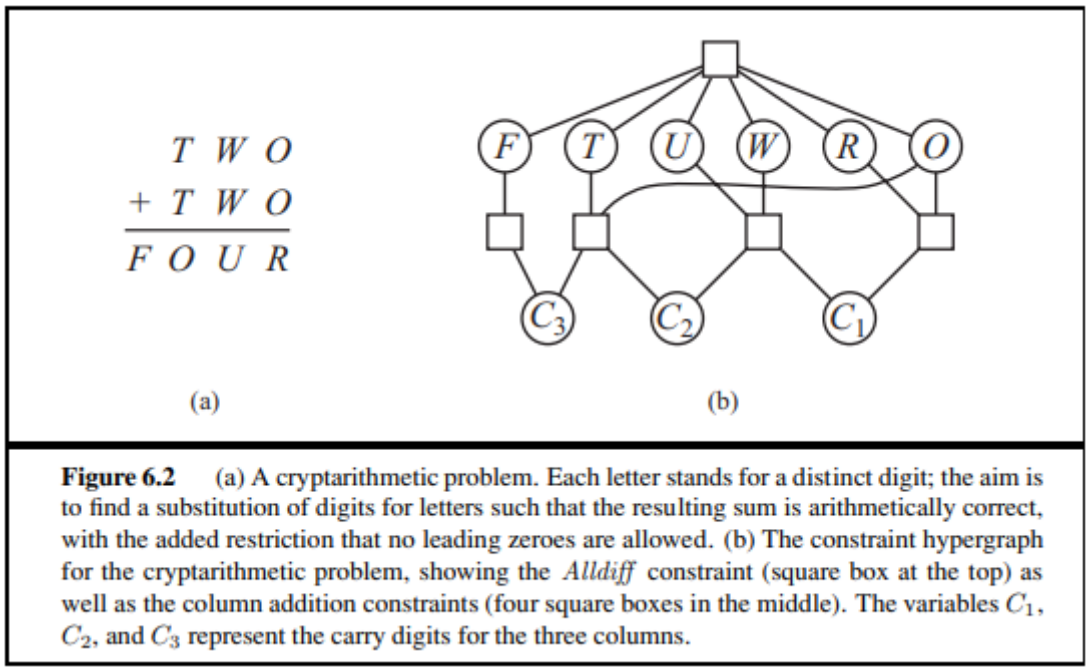


Figure 6.2 (a) A cryptarithmetic problem. Each letter stands for a distinct digit; the aim is to find a substitution of digits for letters such that the resulting sum is arithmetically correct, with the added restriction that no leading zeroes are allowed. (b) The constraint hypergraph for the cryptarithmetic problem, showing the *Alldiff* constraint (square box at the top) as well as the column addition constraints (four square boxes in the middle). The variables C_1, C_2, and C_3 represent the carry digits for the three columns.

Şekil 6.2. (a) Her harfin farklı bir rakamı temsil ettiği, başta sıfırın yasak olduğu aritmetik bulmacası. (b) En üstteki kare Alldiff genel kısıtını; ortadaki dört kare sütun toplama kısıtlarını; C_1, C_2, C_3 elde rakamlarını gösteren kısıt hipergrafı.

Sütunların aritmetik kısıtları elde değişkenleriyle yazılır: $O+O=R+10C_1$; $C_1+W+W=U+10C_2$; $C_2+T+T=O+10C_3$; $C_3=F$. Hipergraf, normal değişken düğümlerinin (daireler) yanında birçok değişkeni bir arada içeren kısıtları temsil eden hiperdüğümlerden (kareler) oluşur.

> **Adım adım geçerli çözüm:** O=4 seçersek 4+4=8 → R=8, C_1=0. W=3 seçersek 3+3=6 → U=6, C_2=0. T=7 seçersek 7+7=14 → O=4, C_3=1. Son elde F=1 olur. Böylece **734+734=1468**; 7,3,4,1,6,8 rakamlarının tümü farklıdır ve ilk basamak sıfır değildir.

Yeterli yardımcı değişken eklenirse her sonlu değer kümeli n'li kısıt, ikili kısıtlara indirgenebilir. **Dual graph (ikili/dual grafik) dönüşümü** başka bir yöntemdir: Orijinal grafiğin her kısıtı yeni grafikte bir değişkene dönüştürülür; ortak değişkenleri bulunan orijinal kısıt çiftleri arasında uyumluluk ilişkisi tanımlanır. Örneğin X+Y=5 ve X<Z kısıtlarının olası kombinasyonları X değeri bakımından uyuşmak zorundadır.

Her şeyi ikili kısıtlarla yazabilmemize rağmen Alldiff gibi genel kısıtlar tercih edilir: (1) tanımı daha kolay ve hataya daha az açıktır; (2) bu kısıtlar için daha güçlü özel çıkarım algoritmaları geliştirilebilir.

Kesin kısıt ve tercih kısıtı; COP

Buraya kadar bütün kısıtlar kesin (hard) kısıtlardı; ihlal edildiklerinde çözüm geçersiz olur. Oysa gerçek uygulamalarda **tercih kısıtları (preference/soft constraints)** da bulunur. Örneğin bir öğretim üyesi aynı anda iki ders veremez (kesin); ancak sabah dersi tercih edebilir (tercih). Öğleden sonra atanması geçerli fakat daha az istenir olabilir.

Tercihler atamalara maliyet vererek ifade edilir. Sabah için maliyet=1, öğleden sonra için maliyet=2 ise toplam maliyeti en aza indiren geçerli ders programı aranır. Böyle probleme **Constraint Optimization Problem (COP — Kısıt Optimizasyon Problemi)** denir. Yol tabanlı veya yerel optimizasyonla ve doğrusal programlama gibi yöntemlerle ele alınabilir.

CSP: bütün kesin kısıtları sağlayan bir çözüm bul.
COP: bütün kesin kısıtları sağlayan çözümler arasında maliyeti en aza indir.

6.2 Kısıt Yayılımı: CSP'lerde Çıkarım

Geleneksel durum uzayı aramasında algoritmanın temel işi aramaktır. CSP'de ise algoritma yeni atamalar arasında arama yapabilir veya **kısıt yayılımı (constraint propagation)** adı verilen özel bir çıkarım gerçekleştirebilir. Kısıt yayılımı bir değişkenin yasal değerlerini azaltır; bu azalma diğer değişkenlerin seçeneklerini de daraltabilir. Bu işlem aramayla iç içe yürütülebilir veya ön işleme olarak, arama başlamadan yapılabilir. Kimi zaman bütün problem yalnızca çıkarımla çözülür.

Temel kavram **yerel tutarlılıktır (local consistency)**. Değişkenleri düğüm, ikili kısıtları yay olarak gördüğümüzde grafiğin parçalarında yerel tutarlılık sağlayarak tutarsız değerleri ağ boyunca eleyebiliriz. Yerel tutarlılığın çeşitli seviyeleri aşağıda açıklanmıştır.

6.2.1 Düğüm tutarlılığı (Node Consistency)

Bir değişken, değer kümesindeki bütün değerler kendi üzerindeki tekli kısıtları sağlıyorsa **düğüm-tutarlıdır**. Avustralya örneğinde SA bölgesi yeşil olamıyorsa başlangıç {kırmızı,yeşil,mavi} değer kümesinden yeşil elenir ve {kırmızı,mavi} kalır. Bütün düğümler bu koşulu sağlıyorsa ağ düğüm-tutarlıdır. Tekli kısıtların etkisi değer kümelerine yansıtılarak ayrı kısıtlar kaldırılabilir.

> **Örnek:** D_x={1,2,3,4,5} ve X>3 ise yalnızca {4,5} kalır. Bu işlem Y gibi başka değişkenleri içermez; bu nedenle düğüm tutarlılığıdır.

6.2.2 Yay tutarlılığı (Arc Consistency) ve AC-3

Bir X_i değişkeni, X_j değişkenine göre yay-tutarlıdır; eğer X_i'nin mevcut değer kümesindeki **her x değeri** için X_j'nin değer kümesinde ilgili ikili kısıtı karşılayan **en az bir y** bulunuyorsa. Bütün yönlü yaylarda bu özellik sağlanıyorsa ağ yay-tutarlıdır.

Her $x \in D_i$ için en az bir $y \in D_j$ bulunmalı ve $C_{ij}(x,y)$ doğru olmalı.

Örneğin X ve Y'nin ikisi de 0–9 rakamlarından değer alsın ve $Y=X^2$ kısıtı bulunsun. İzin verilen çiftler yalnızca (0,0), (1,1), (2,4), (3,9)'dur. X'in değer kümesi {0,1,2,3}; Y'ninki {0,1,4,9} olur. Çünkü X=4 seçilirse Y=16 gerekir ve 16 bir rakam değildir.

Öte yandan komşu iki Avustralya bölgesinin üç renkli SA≠WA kısıtında her renge karşılık karşı tarafta en az iki farklı renk vardır. Dolayısıyla yay tutarlılığı tek başına hiçbir değeri azaltmaz; yay-tutarlı olmak problem çözüldü demek değildir.

AC-3 algoritması (Şekil 6.3)

AC-3, kısıt grafiğindeki yayları bir kuyrukta tutar. Başlangıçta bütün yönlü yaylar kuyruğa alınır. (X_i,X_j) çıkarılınca X_i'nin X_j'ye göre desteklenmeyen değerleri silinir. D_i değişmediyse sıradaki yaya geçilir. D_i küçülürse X_i'nin X_j dışındaki komşularından X_i'ye gelen (X_k,X_i) yayları yeniden kuyruğa alınır; çünkü yeni daralma diğer alanları etkileyebilir. Herhangi bir değer kümesi boşalırsa problem çözümsüzdür. Kuyruk boşalınca bütün yaylar tutarlıdır, kalan CSP'nin çözümleri orijinaliyle aynıdır.

```
AC-3(CSP):
    kuyruk ← CSP'deki bütün yönlü yaylar
    kuyruk boş değilken:
        (Xi, Xj) ← kuyruktan çıkar
        REVISE(Xi, Xj) bir değer sildiyse:
            Di boşsa: başarısız dön
            Xi'nin Xj dışındaki her komşusu Xk için:
                (Xk, Xi) yayını kuyruğa ekle
    başarı dön

REVISE(Xi, Xj):
değişti ← yanlış
Di'deki her x için:
Dj'de x'i destekleyen bir y yoksa:
x'i Di'den sil; değişti ← doğru
değişti değerini döndür
```

Çalışma mantığı: AC-3 sadece bir düğümdeki değişimi kaydetmez; bu değişimden etkilenebilecek yayları tekrar işleyerek sonuçları ağ boyunca yayar. Bir CSP için c ikili yay, değişken başına en çok d değer varsa bir yay en fazla d kez kuyruğa yeniden girebilir. Her yay kontrolü $O(d^2)$ olduğundan standart en kötü durum karmaşıklığı **$O(cd^3)$**'tür. AC-4 için $O(cd^2)$ sınırı bulunsa da ortalama örneklerde AC-3'ten yavaş olabilir.

> **AC-3 sayısal izleme:** $D_a=D_\beta=D_c=\{1,2,3\}$, A<B ve B<C olsun. (A,B) kontrolü A=3'ü eler: A={1,2}. (B,A) ile B=1 elenir: B={2,3}. (B,C) ile B=3 elenir: B={2}. (C,B) ile C={3} olur. Yeniden (A,B) kontrolünde A=2 elenir: A={1}. Sonuç A=1, B=2, C=3; hiçbir tahmine gerek kalmadı.

Genelleştirilmiş yay tutarlılığı (Generalized Arc Consistency)

Yay tutarlılığı ikiden fazla değişken içeren kısıtlara genişletilebilir. X_i'nin değer kümesindeki her v için, n'li kısıtı sağlayan ve diğer değişkenlerin güncel değer kümelerinden gelen en az bir değer grubu bulunmalıdır. Örneğin X<Y<Z ve üç değişken için de {0,1,2,3} değer kümesi varsa X=2 veya X=3 desteklenemez; çünkü X'ten daha büyük art arda iki değer seçilemez. Bu değerler X'in kümesinden kaldırılır.

6.2.3 Yol tutarlılığı (Path Consistency)

Yay tutarlılığı birçok alanda işe yarasa da her tutarsızlığı belirleyemez. Örneğin Avustralya haritasını yalnızca kırmızı ve maviyle boyamayı düşünelim. Her komşu çift birbirini iki renk açısından destekler; yani ağ yay-tutarlıdır. Buna rağmen WA, NT ve SA birbirleriyle komşu olduğundan üçünün iki renkle farklı boyanması mümkün değildir.

Yol tutarlılığı üçüncü bir değişken üzerinden ikili kısıtları sıkılaştırır. $\{X_i,X_j\}$ için tutarlı olan her (a,b) değer çiftine, aradaki X_m değişkeninde hem X_i-X_m hem X_m-X_j kısıtlarını sağlayan bir değer atanabilmelidir. Böyle destek bulunmayan ikili kombinasyon elenir.

WA ve SA'ya yalnızca iki renkten farklı olanlar atanabilir: (kırmızı,mavi) ve (mavi,kırmızı). NT, her iki bölgeye de komşudur. Birinci durumda NT ne kırmızı ne mavi, ikinci durumda da ne mavi ne kırmızı olabilir. Dolayısıyla iki atama da elenir; problemde hiçbir çözüm yoktur. PC-2 algoritması bu tür yol tutarlılığını AC-3'e benzer şekilde gerçekleştirir.

6.2.4 k-tutarlılığı (K-Consistency)

Bir CSP, herhangi k−1 değişkene yapılmış tutarlı bir atama seçilen herhangi k'ncı değişkene tutarlı bir değer verilerek genişletilebiliyorsa **k-tutarlıdır**. 1-tutarlılığı düğüm, 2-tutarlılığı yay, ikili kısıt ağlarında 3-tutarlılığı yol tutarlılığıyla aynı kavramdır.

Güçlü k-tutarlılığı (strongly k-consistent), yalnızca k seviyesinde değil, 1'den k'ya kadar bütün seviyelerde tutarlı olma koşuludur. n değişkenli bir ağ güçlü n-tutarlı hâle getirildiğinde X_1'den X_n'ye kadar değerleri geri izleme olmadan seçebiliriz. Bu sonraki değer atama aşamasının maliyeti $O(n^2d)$ ile sınırlanır. Fakat ağı güçlü n-tutarlı hâle getirmek en kötü durumda n'ye göre üstel zaman ve üstel bellek gerektirir. Bu nedenle pratikte genellikle 2-tutarlılığı, daha seyrek olarak 3-tutarlılığı kullanılır.

Seviye	İlke	Karşılığı
1	Bir değişkenin tekli kısıtları	Node consistency
2	Her değere komşuda destek bulunması	Arc consistency
3	İkili atamaya üçüncü değişken desteği	Path consistency (ikili CSP)
k	k−1 tutarlı atamayı k'ncıya genişletme	k-consistency

6.2.5 Genel kısıtlar (Global Constraints)

Çok sayıda değişkene uygulanan genel kısıtlar için özel çıkarım algoritmaları, ikili kısıtlara tek tek uygulanan yöntemlerden daha etkili olabilir. Alldiff($X_1,\ldots,X_m$), bütün değişkenlerin farklı değer almasını gerektirir. m değişkenin kullanabileceği toplam farklı değer sayısı n ise **m>n** durumunda çözüm imkânsızdır.

Basit Alldiff yayılımında tek elemanlı değer kümesine sahip her değişken bulunur; bu değer diğer değişkenlerin kümelerinden kaldırılır; işlem tekrarlanır. Herhangi bir küme boşalırsa veya değişken sayısı kullanılabilir farklı değer sayısını aşarsa tutarsızlık tespit edilir. Örneğin WA=kırmızı ve NSW=kırmızı ise SA, NT ve Q için yalnızca {yeşil,mavi} kalır. Üç değişken ve iki renk Alldiff'i ihlal eder; ikili yay tutarlılığı buna tek başına ulaşamayabilir.

Kaynak kısıtları (Resource / Atmost)

Dört göreve atanan çalışan sayıları $P_1,\ldots,P_4$ olsun. En çok 10 kişi kullanılabileceği Atmost(10,P_1,P_2,P_3,P_4) ile yazılır; eşdeğeri $P_1+P_2+P_3+P_4 \le 10$'dur. Her değişkenin değer kümesi {3,4,5,6} ise minimum toplam 3+3+3+3=12 olur: çözüm yoktur. Değer kümeleri {2,3,4,5,6} ise bir değişken 5 olursa diğerlerinin minimumu 2+2+2 ve toplam 11 olur; bu nedenle 5 ve 6 bütün kümelerden elenir ve {2,3,4} kalır.

Sınır yayılımı (Bounds Propagation)

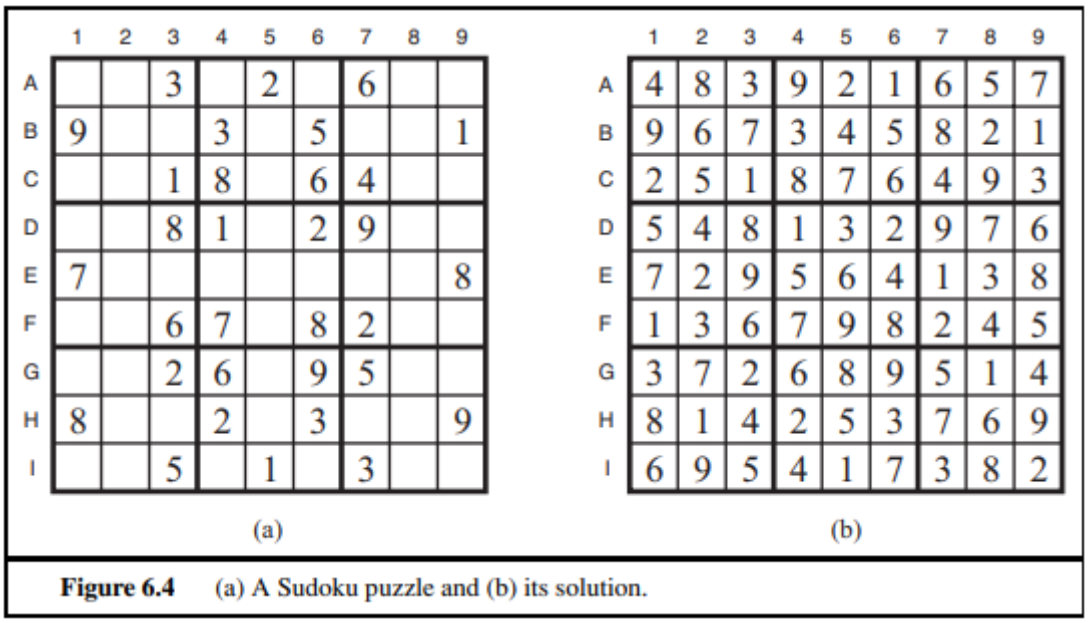
Büyük sayısal kaynak problemlerinde her tam sayıyı tek tek saklamak yerine değişkenlerin alt ve üst sınırlarını tutmak uygundur. Örneğin iki uçuşta kapasite $F_1 \in [0,165]$ ve $F_2 \in [0,385]$ olsun. Birlikte 420 kişi taşınacaksa $F_1+F_2=420$ kısıtı bulunur. F_2 en çok 385 olduğundan F_1 en az 420−385=35; F_1 en çok 165 olduğundan F_2 en az 420−165=255 kişi taşımalıdır. Yeni aralıklar:

$$F_1 \in [35,165] \quad F_2 \in [255,385]$$

Alt ve üst sınırların ilgili diğer değişkenlerde desteklenmesi koşulu **sınır tutarlılığı (bounds consistency)** olarak adlandırılır. Sınır yayılımı pratik çizelgeleme ve lojistik CSP'lerinde yaygın kullanılır.

6.2.6 Sudoku örneği (Sudoku Example)

Sudoku, milyonlarca kişiyi farkında olmadan CSP ile tanıştırmıştır. Tahta 81 hücreden oluşur; bazı hücrelere başlangıçta 1–9 rakamları yerleştirilmiştir. Boş hücreler doldurulurken bir satır, sütun veya 3×3 kutu içerisinde hiçbir rakam iki kez bulunmamalıdır. Bu grupların her birine **birim (unit)** denir. Geleneksel yayımlanan Sudoku bulmacalarının tek çözümü bulunur; CSP çözücüleri kolay örnekleri son derece hızlı çözebilir.



	1	2	3	4	5	6	7	8	9
A			3		2		6		
B	9			3		5			1
C			1	8		6	4		
D			8	1		2	9		
E	7								8
F			6	7		8	2		
G			2	6		9	5		
H	8			2		3			9
I			5		1		3		

(a)

	1	2	3	4	5	6	7	8	9
A	4	8	3	9	2	1	6	5	7
B	9	6	7	3	4	5	8	2	1
C	2	5	1	8	7	6	4	9	3
D	5	4	8	1	3	2	9	7	6
E	7	2	9	5	6	4	1	3	8
F	1	3	6	7	9	8	2	4	5
G	3	7	2	6	8	9	5	1	4
H	8	1	4	2	5	3	7	6	9
I	6	9	5	4	1	7	3	8	2

(b)

Figure 6.4 (a) A Sudoku puzzle and (b) its solution.

Şekil 6.4. (a) Sudoku bulmacası. (b) Aynı bulmacanın çözümü. Satırlar A–I, sütunlar 1–9 olarak gösterilir.

Sudoku CSP'sinde 81 değişken (A1,...,I9) vardır. Boş hücrelerin değer kümesi {1,...,9}, verilmiş hücrelerin kümesi tek bir rakamdır. 27 Alldiff kısıtı bulunur: 9 satır, 9 sütun, 9 kutu. Örneğin Alldiff(A1,A2,...,A9) ilk satırı; Alldiff(A1,B1,...,I1) ilk sütunu; Alldiff(A1,A2,A3,B1,B2,B3,C1,C2,C3) sol üst kutuyu temsil eder.

Şekil 6.4 üzerinden AC-3 çıkarımı

Alldiff kısıtlarını A1≠A2 gibi ikili kısıtlara açtığımızı varsayalım. Şekil 6.4(a)'da E6 boş hücresi orta 3×3 kutuda yer alır. Bu kutudaki 1,2,7,8 rakamları E6 için yasaktır. E6'nın sütununda ise 5,6,2,8,9,3 bulunur. İki eleme birlikte E6 için yalnızca 4'ü bırakır:

$$D(E6) = \{1,2,3,4,5,6,7,8,9\} - \{1,2,7,8\} - \{5,6,2,8,9,3\} = \{4\}$$

E6=4 olduktan sonra altıncı sütunda eksik rakamlar {1,7}'dir. I6 ile aynı satırda I5=1 bulunduğundan I6≠1; dolayısıyla I6=7 olur. Bu sefer altıncı sütunda yalnızca A6 boş kalır; A6=1 olmak zorundadır. Bir çıkarımın sonraki hücreleri zorlaması, kısıt yayılımının zincirleme etkisidir.

Adım	Bilgi	Çıkarım
1	Orta kutu + altıncı sütun	E6 = 4
2	I6 ∈ {1,7} ve I5=1	I6 = 7
3	Altıncı sütundaki son boşluk	A6 = 1

Bu Sudoku örneğinde AC-3 devam ettirildiğinde bütün hücreler tek elemanlı hâle getirilebilir. Ancak AC-3 yalnızca kolay Sudokularda yeterlidir. Daha zorlarda PC-2 daha fazla çıkarım yapabilir; kitap bir Sudoku için 255.960 farklı yol kısıtının değerlendirilebileceğini belirtir. En zor bulmacalar daha gelişmiş stratejiler gerektirir.

Çıplak üçlüler (Naked Triples)

Aynı birimde üç hücrenin değer kümeleri {1,8}, {3,8}, {1,3,8} ise bu üç hücrenin, hangi sırada oldukları bilinmese de 1,3,8 rakamlarını aralarında paylaşmaları gerekir. Dolayısıyla aynı birimdeki diğer bütün hücrelerden 1,3,8 elenebilir. Örneğin dördüncü hücrenin adayları {1,3,5,8,9} ise {5,9} kalır. Bu strateji aslında Sudoku'ya özel değildir; Alldiff kısıtındaki çıkarımın bir uygulamasıdır.

Bölümün kilit fikri: Problemin özelliklerini değişken, alan ve kısıt olarak ifade ettikten sonra düğüm/yay/yol tutarlılığı ve genel kısıt çıkarımları birçok farklı CSP'ye aynı biçimde uygulanabilir.

6.3 CSP'ler İçin Geri İzlemeli Arama (Backtracking Search)

Sudoku gibi problemler kimi zaman kısıt çıkarımıyla çözülebilse de birçok CSP için yalnızca çıkarım yeterli değildir. Bir noktada değer atamalarını deneyerek çözümü aramak gerekir. Bu bölümde **kısmi atamalar üzerinde** çalışan geri izlemeli aramayı, sonraki bölümde ise tam atamalar üzerinde yerel aramayı ele alacağız.

3. bölümdeki derinlik sınırlı arama doğrudan CSP'ye uygulanabilir: her durum kısmi atamadır; eylem, yeni bir değişkene değer atamaktır. Ancak n değişken ve her biri d değer aldığında başlangıçta herhangi n değişkenden herhangi d değer seçilebilir; dallanma faktörü nd olur. Sonraki seviyede (n−1)d ve benzeri şekilde devam eder. Sonunda $\mathbf{n!\cdot d^n}$ yaprak oluşur, oysa yalnızca $\mathbf{d^n}$ farklı tam atama vardır.

Bu fazlalığın nedeni, aynı atamaya farklı sıralardan ulaşılmasıdır. CSP'lerde **değişme özelliği (commutativity)** vardır: A=1 sonra B=2 yapma ile B=2 sonra A=1 yapma aynı kısmi atamayı üretir. Bu nedenle arama ağacındaki her düğümde yalnızca **bir değişken** seçilip değerlerine göre dallanmak yeterlidir. Böylece yaprak sayısı d^n düzeyine iner.

> **Küçük sayısal örnek:** n=3 değişken, d=2 değer için naif aramanın yaprak sayısı $3! \times 2^3 = 6 \times 8 = 48$'dir. Farklı tam atama sayısı sadece $2^3 = 8$'dir. Değişken seçimini her seviyede sabitlemek, aynı sonucun tekrar tekrar üretilmesini engeller.

Geri izlemeli aramanın tanımı ve Şekil 6.5

Backtracking, her adımda bir değişkene değer atayan **derinlik öncelikli arama** türüdür. Değişkenin bütün geçerli değerleri başarısız olursa önceki karar noktasına dönülür. Algoritma henüz atanmamış bir değişken seçer, onun değerlerini belirli sırayla dener, tutarlılık kontrolü ve isteğe bağlı çıkarım uygular. Çelişki durumunda yapılan atamalar/çıkarımlar geri alınır ve başka değer denenir.

```
BACKTRACKING-SEARCH(csp):
    BACKTRACK({}, csp) sonucunu döndür

BACKTRACK(assignment, csp):
assignment tamamsa: assignment döndür
var ← SELECT-UNASSIGNED-VARIABLE(csp)
ORDER-DOMAIN-VALUES(var) sırasındaki her value için:
value mevcut atamayla tutarlıysa:
{var=value} atamasını ekle
inferences ← INFERENCE(csp, var, value)
inferences başarısız değilse:
çıkarımları uygula
result ← BACKTRACK(assignment, csp)
result başarısız değilse: result döndür
var atamasını ve çıkarımları geri al
başarısız dön
```

Şekil 6.5 açıklaması: Algoritma özyinelemeli DFS'ye dayanır. SELECT-UNASSIGNED-VARIABLE ve ORDER-DOMAIN-VALUES, genel amaçlı sezgisellerle özelleştirilebilir. INFERENCE yordamı yay/yol/k-tutarlılığını isteğe bağlı uygulayabilir. Atama başarısız olduğunda hem açık atama hem çıkarımla yapılan değişiklikler geri alınmalıdır. CSP standart gösterime sahip olduğundan ayrıca probleme özel başlangıç durumu, geçiş modeli ve hedef testi tanımlamak zorunlu değildir.

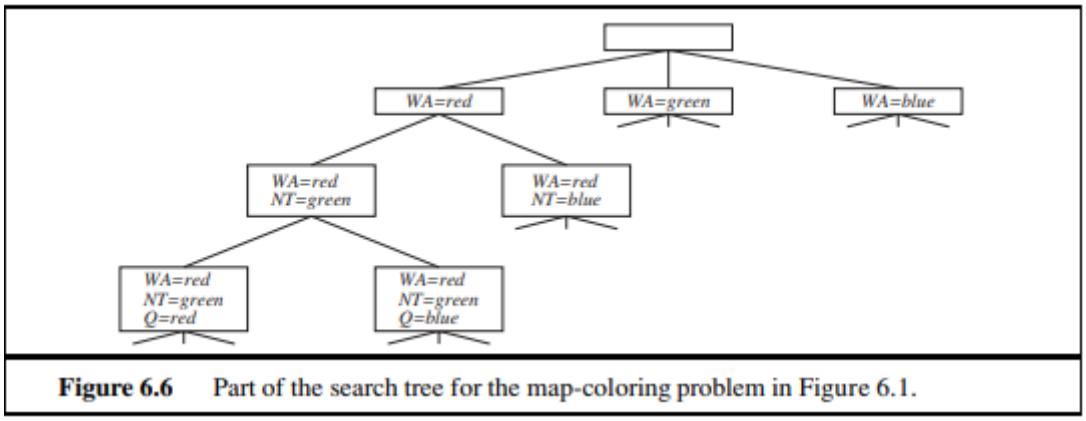


Figure 6.6 Part of the search tree for the map-coloring problem in Figure 6.1.

Şekil 6.6. Şekil 6.1'deki harita boyama probleminin arama ağacından bir bölüm. Değişkenlerin sırası WA, NT, Q, … olarak seçilmiştir.

Şekil 6.6'da WA için kırmızı, yeşil ve mavi dalları bulunur. WA=kırmızı seçilince NT'nin kırmızı olmasına izin verilmez; NT=yeşil veya NT=mavi denenir. WA=kırmızı ve NT=yeşil sonrasında Q kırmızı ya da mavi olabilir. Q=mavi seçilirse SA'nın kırmızı, yeşil ve mavi komşuları bulunduğundan hiçbir seçeneği kalmaz. Bu dal budanır ve önceki karara geri dönülür.

CSP için verimli genel amaçlı arama geliştirmek üzere üç soruya odaklanırız: (1) Sırada hangi değişken seçilmeli, değerler hangi sırayla denenmeli? (2) Arama boyunca ne tür çıkarımlar yapılmalı? (3) Çakışmadan sonra aynı başarısızlığa tekrar nasıl düşülmemeli?

6.3.1 Değişken ve değer sıralaması (Variable and Value Ordering)

Minimum Remaining Values (MRV): En az kalan değer

Atanmamış değişkenleri $X_1, X_2, \ldots$ sabit sırasıyla seçmek genellikle verimli değildir. Şekil 6.6'da WA=kırmızı ve NT=yeşil atandıktan sonra SA için yalnızca mavi kalır. Bu yüzden Q'dan önce SA=mavi yapmak daha uygundur; ardından Q, NSW ve V'nin renkleri zorunlu hâle gelebilir.

MRV (Minimum Remaining Values), kalan geçerli değer sayısı en az olan değişkeni seçer; “most constrained variable” veya “fail-first” olarak da bilinir. Başarısızlık olasılığı yüksek değişkeni erken seçerek gereksiz aramayı budar. Bir değişkenin hiçbir geçerli değeri kalmamışsa MRV onu seçer ve çelişki anında görülür. MRV rastgele veya statik sıralamaya kıyasla çok daha verimli olabilir; kitap bazı problemlerde 1.000 kat ve üzeri fark gözlendiğini belirtir (probleme göre değişir).

MRV: X^* = en küçük $|D_i|$ değerine sahip atanmamış X_i

Değişken	Kalan değerler	Adet
A	{1,2,3,4}	4

Değişken	Kalan değerler	Adet
B	{1,3}	2
C	{2}	**1 ← MRV**
D	{1,2,3}	3

Degree Heuristic: Derece sezgiseli

Avustralya haritasında başlangıçta bütün değişkenlerin üç rengi olduğu için MRV ilk seçimde eşitlik oluşturur. **Derece sezgiseli**, henüz atanmamış diğer değişkenlerle en fazla kısıtı bulunan değişkeni tercih eder; böylece sonraki dallanma sayısını düşürmeyi amaçlar. SA'nın derecesi 5, T'nin derecesi 0'dır; SA ilk sırada seçilebilir. MRV genellikle daha güçlüdür, Degree ise eşitlikleri kırmak için kullanılabilir. SA'yı ilk seçen derece yaklaşımı bu özel harita problemini geri izlemesiz çözmeyi sağlayabilir.

Least Constraining Value (LCV): En az kısıtlayıcı değer

Değişken seçildikten sonra değerler hangi sırayla denenecek? **LCV**, komşu değişkenlerin değer kümelerinden **en az seçeneği ortadan kaldıran** değeri önce dener. WA=kırmızı ve NT=yeşil iken sıradaki Q değişkeni olsun. SA'nın tek kalan rengi mavi olduğundan Q=mavi seçilirse SA'nın değeri kalmaz. Q=kırmızı seçimi SA'ya mavi bırakır; LCV kırmızıyı önce dener.

Neden MRV “fail-first”, LCV “fail-last”? MRV, en sorunlu değişkeni erkenden seçerek büyük başarısız dalları budar. LCV ise tek bir çözüm bulmak istediğimizde diğerlerine esneklik bırakan değeri erken deneyip başarılı dala ulaşmayı hedefler. Bütün çözümler listelenecekse değer sıralaması tüm değerlerin incelenmesi gereğini değiştirmez.

Sezgisel	Ne seçer?	Temel ölçüt
MRV	Değişken	En az geçerli değeri kalan
Degree	Değişken	Atanmamış komşularla en fazla kısıtı olan
LCV	Değer	Komşularda en az değer kaybı yaratan

6.3.2 Arama ve çıkarımın iç içe kullanılması

AC-3, aramadan önce değer kümelerini daraltabilir. Fakat arama sırasında bir değişkene yeni değer atamak da yeni çıkarımlar üretir. En basit yöntem **Forward Checking (İleri Kontrol)**'dir. X=x atandığında X ile kısıt bağlantısı bulunan **atanmamış komşu Y** değişkenlerinin değer kümelerinden X=x ile tutarsız bütün değerler silinir. Böylece daha sonra denenmesi gerekecek başarısız değerler erkenden elenir.

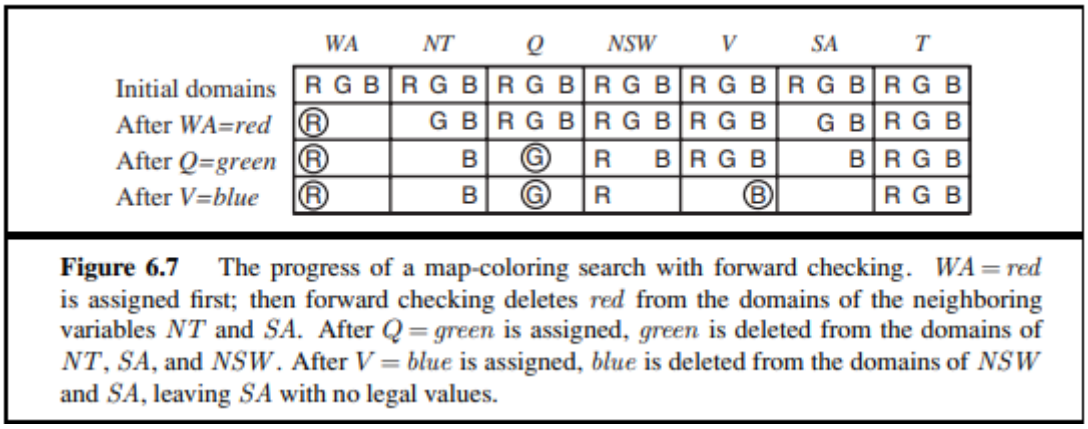


Figure 6.7 The progress of a map-coloring search with forward checking. $WA = red$ is assigned first; then forward checking deletes red from the domains of the neighboring variables NT and SA. After $Q = green$ is assigned, $green$ is deleted from the domains of NT, SA, and NSW. After $V = blue$ is assigned, $blue$ is deleted from the domains of NSW and SA, leaving SA with no legal values.

Şekil 6.7. Forward Checking ile harita boyama ilerleyişi. WA=kırmızı sonrası NT ve SA'dan kırmızı; Q=yeşil sonrası NT, SA ve NSW'den yeşil; V=mavi sonrası NSW ve SA'dan mavi elenir. SA'nın değer kümesi boşalınca başarısızlık anlaşılır. R/G/B: kırmızı/yeşil/mavi.

Şekil 6.7'nin dört aşaması

Değişken	Başlangıç	WA=R sonrası	Q=G sonrası	V=B sonrası
WA	RGB	R	R	R
NT	RGB	GB	B	B
Q	RGB	RGB	G	G
NSW	RGB	RGB	RB	R
V	RGB	RGB	RGB	B
SA	RGB	GB	B	Ø
T	RGB	RGB	RGB	RGB

Birinci önemli nokta: WA=R ve Q=G atamalarından sonra NT ve SA'nın kümeleri tek bir renk olan maviye düşer; bazı dallanmalar ortadan kalkar. **İkinci nokta:** V=B ataması SA'nın son mavi değerini de siler; algoritma hemen geri döner. Forward Checking ile MRV birlikte kullanıldığında daralmış kümelerde az seçeneği olan NT veya SA erken işlenir. İleri kontrol, MRV'nin kullanacağı alan boyutu bilgisini her adımda günceller.

MAC: Maintaining Arc Consistency

Forward Checking bütün tutarsızlıkları yakalayamaz; çünkü yalnızca **atanmış değişkenden atanamamış komşulara** bakar. Şekil 6.7'de WA=R ve Q=G sonrasında NT=B ve SA=B kalır. Oysa NT ile SA komşudur ve iki bölge aynı mavi rengi alamaz. Forward Checking, birbirine komşu bu iki henüz atanmamış değişkeni tekrar karşılaştırmadığından çelişkiyi göremez.

MAC (Maintaining Arc Consistency), yeni atama X_i=v yapıldıktan sonra AC-3'ü yeniden işletir. İlk kuyrukta X_i'nin henüz atanmamış komşularından X_i'ye gelen (X_j,X_i) yayları bulunur; buradan yayılım diğer yaylara devam eder. Herhangi değer kümesi boşalırsa başarısızlık hemen bildirilir. MAC, ilk komşu kontrollerini yapmanın yanı sıra yeni daralmaların etkisini yinelemeli yaydığı için Forward Checking'den daha güçlüdür. Daha fazla hesaplama gerektirebilir.

Önemli nüans: Yalnızca arama *öncesinde* bir kez AC-3 yapmak, daha sonra atanan değerlerin doğurduğu yeni kısıt etkilerini otomatik olarak ele almaz. Bu nedenle ön işlem ile arama sırasında yapılan Forward Checking/MAC birbirinin her durumda tam alternatifi değildir.

6.3.3 Akıllı geri izleme: Geriye bakış

Kronolojik geri izleme ve Backjumping

Basit BACKTRACK algoritması başarısızlık durumunda **en son atanmış değişkene** döner; bu **kronolojik geri izleme**dir. Fakat başarısızlığın nedeni daha önceki bir değişken olabilir. Örneğin değişken sırası Q, NSW, V, T, SA, WA, NT olsun. {Q=kırmızı, NSW=yeşil, V=mavi, T=kırmızı} sonrasında SA için hiçbir renk kalmaz; çünkü Q, NSW ve V üç rengi kullanmıştır. Kronolojik yöntem Tazmanya'nın rengini değiştirir. Oysa T bölgesi SA'yla komşu değildir; hiçbir faydası yoktur.

Backjumping (geri sıçrama), başarısızlığa neden olan atamaları izler. SA'nın **çakışma kümesi (conflict set)** bu durumda {Q=kırmızı, NSW=yeşil, V=mavi}'dir. Bu kümede en son atanmış değişken V olduğundan T atlanır, doğrudan V'ye dönülür. Forward Checking bir atama nedeniyle başka değişkenin değerini elediğinde ilgili atamayı o değişkenin çakışma kümesine kaydederek bu bilgiye katkı sağlayabilir.

Ancak basit Backjumping'in başarısızlığı anladığı birçok dal, Forward Checking ile daha önce zaten budanır. Bu nedenle Forward Checking ya da MAC ile birlikte basit Backjumping çoğunlukla gereksizdir. Gene de başarısızlığın nedenini izleme düşüncesi daha gelişmiş bir geri izleme türüne yol açar.

Conflict-Directed Backjumping (CBJ)

Bir dal, henüz herhangi bir tek değişkenin değer kümesi boşalmadan da başarısız olmaya mahkûm olabilir. Örneğin WA=kırmızı ve NSW=kırmızı ataması, SA-NT-Q üçgeninde yalnızca yeşil ve maviyi bıraktığından birlikte çözümsüzdür. Daha sonra T=kırmızı atamak, ardından NT, Q, V, SA üzerinde arama yapmak bu asıl nedeni değiştirmez.

Çakışma yönlendirmeli geri sıçrama, bir değişkenin yalnızca doğrudan çakışmalarını değil, **kendisi ve sonrasındaki değişkenlerin neden birlikte çözülemediğini** de izler. Bir dalın son başarısızlığında SA için conf(SA)={WA,NT,Q} olsun. En son Q'ya dönülür; Q'nun mevcut conf(Q)={NT,NSW} kümesiyle SA'nın Q dışındaki elemanları birleştirilir: conf(Q)={WA,NT,NSW}. Sonra NT'ye sıçranır; conf(NT)={WA} ile Q kümesinin NT dışındakileri birleşir: conf(NT)={WA,NSW}. Böylece NSW'ye geri dönülür ve ilgisiz T atlanır.

X_j başarısızsa, çakışma kümesindeki en son X_i'ye geri sıçra:
$\text{conf}(X_i) \leftarrow \text{conf}(X_i) \cup (\text{conf}(X_j) - \{X_i\})$

Kısıt öğrenme (Constraint Learning) ve No-Good

Backjumping nereye dönmek gerektiğini söyler; fakat aynı başarısız kombinasyonla gelecekte yeniden karşılaşmayı da istemeyiz. **Kısıt öğrenme**, başarısızlığa neden olan atama kombinasyonlarını kaydeder. Böyle bir geçersiz kombinasyon **no-good** olarak adlandırılır; yeni bir kısıt biçiminde CSP'ye eklenebilir ya da önbellekte tutulabilir.

Şekil 6.6'da {WA=kırmızı, NT=yeşil, Q=mavi} ataması SA'ya renk bırakmadığı için no-good'dur. Tek bir dallanma sırası varsa tekrar görülmeyeceğinden saklamak fazla yarar sağlamayabilir. Ama büyük arama ağacında önce V ve T'nin çeşitli renkleri deneniyorsa aynı üçlü kombinasyonla tekrar karşılaşılabilir. O zaman no-good kaydı çok yararlıdır.

Yasak kombinasyon: ¬(WA=kırmızı ∧ NT=yeşil ∧ Q=mavi)

Modern CSP çözücülerinde kısıt öğrenme, karmaşık problemler için en önemli verimlilik kaynaklarından biridir. Mantık, hatalı tahminleri sadece terk etmek değil, onlardan gelecekte kullanılabilecek bilgi çıkarmaktır.

6.4 CSP'lerde Yerel Arama (Local Search for CSPs)

Yerel arama algoritmaları pek çok CSP'de etkilidir. Önceki bölümde kısmi atamadan ilerlerken, yerel aramada **başlangıç durumu bütün değişkenlere değer atanmış tam bir durumdur**. Daha sonra her adımda yalnızca bir değişkenin değeri değiştirilir. Örneğin sekiz vezir probleminde rastgele yerleştirilmiş sekiz vezirden biri aynı sütunda yeni bir satıra taşınır. İlk durumda birçok kısıtın ihlal edilmesi doğaldır; amaç bu ihlalleri sıfıra indirmektir.

Bir değişkene yeni değer seçerken en açık sezgisel yaklaşım, diğer değişkenlerle **en az sayıda çakışma** oluşturan değeri seçmektir. Buna **Min-Conflicts** denir. Yerel arama, optimizasyon amaçlı COP'lere de genişletilebilir; Hill Climbing ve Simulated Annealing gibi yöntemler amaç fonksiyonunu iyileştirmede kullanılabilir.

Şekil 6.8: Min-Conflicts algoritması

```
MIN-CONFLICTS(csp, max_steps):
    current ← rastgele veya açgözlü tam başlangıç ataması
    1'den max_steps'e kadar:
        current çözümse: current döndür
        var ← çakışma yaşayan değişkenlerden rastgele seç
        value ← var için çakışma sayısını en aza indiren değer
        current içinde var=value yap
    başarısız dön
```

Başlangıç tam ataması rastgele veya her değişken için sırasıyla az çakışma yaratan değeri seçen açgözlü yöntemle kurulabilir. CONFLICTS yordamı, diğer değişkenlerin mevcut değerlerine göre bir değer seçiminin kaç kısıtı ihlal edeceğini sayar. Aynı minimumu sağlayan birden çok değer varsa seçim rastgele yapılabilir.

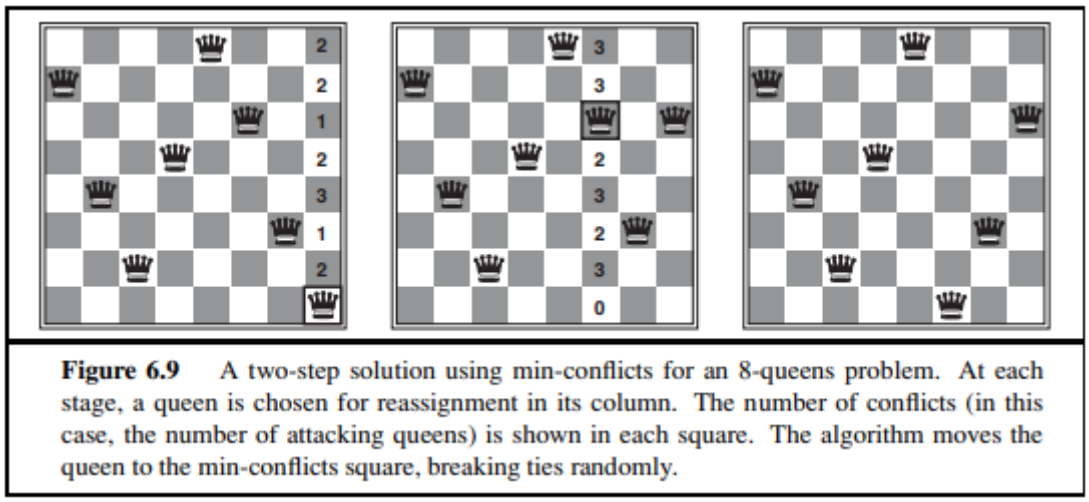


Figure 6.9 A two-step solution using min-conflicts for an 8-queens problem. At each stage, a queen is chosen for reassignment in its column. The number of conflicts (in this case, the number of attacking queens) is shown in each square. The algorithm moves the queen to the min-conflicts square, breaking ties randomly.

Şekil 6.9. Sekiz vezir probleminin Min-Conflicts kullanılarak iki adımda çözülmesi. Her aşamada bir vezir kendi sütununda yeniden yerleştirilir. Karelerdeki sayılar o konumda kaç başka vezirin saldırabileceğini gösterir; eşitlikler rastgele kırılır.

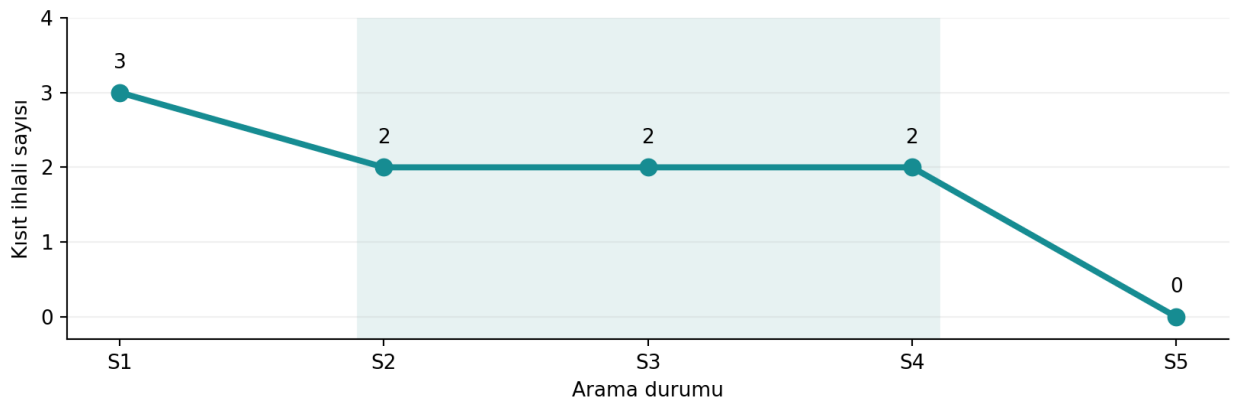
İlk tahtada yerleşim tam fakat çakışmalıdır. Bir vezir seçilir, kendi sütununda en az saldırı alan satıra taşınır. İkinci tahtada başka bir vezir için aynı işlem yapılır ve sıfır çakışmalı kare seçilir. Üçüncü tahtada bütün vezirler farklı satır ve çaprazda olduğundan çözüm elde edilir. Sayılar vezirlerin “tehdit sayısı”dır; çözümde her vezir için sıfırdır.

> **Basit Min-Conflicts hesabı:** A,B,C∈{1,2,3}; bütün değişkenlerin farklı olması gereksin. Başlangıç A=1,B=1,C=2 ise B çakışıyor. B=1 seçimi 1 (A ile) çakışma, B=2 seçimi 1 (C ile) çakışma, B=3 seçimi 0 çakışma oluşturur. B=3 seçildiğinde {1,3,2} çözüm olur.

Min-Conflicts bazı CSP'lerde şaşırtıcı derecede etkilidir. Kitap, n-vezir probleminde ilk yerleşim zamanı hesaba katılmadığında, çözüme ulaşma adım sayısının n'ye yaklaşık bağımsız olduğunu; bir milyon vezirlik deneylerde başlangıçtan sonra ortalama yaklaşık 50 adım gözlendiğini belirtir. Bu deneysel özellik bütün örnekler için kesin garanti değildir. Çözümlerin durum uzayında yoğun bulunması n-veziri yerel arama açısından kolaylaştırır. Hubble Uzay Teleskobu gözlemlerinin planlanmasında haftalık planlama süresinin üç haftadan yaklaşık 10 dakikaya indirildiği tarihsel uygulama örneği de verilir.

Plato, tabu arama ve benzetimli tavlama

Min-Conflicts'in değerlendirme yüzeyi çoğu zaman **plato** alanları içerir: Birbirine komşu birçok tam atamanın aynı sayıda çakışması vardır. Yalnızca kesin iyileştirmeye izin veren arama burada sıkışabilir. **Yatay hareket (sideways move)** aynı puanlı başka bir duruma geçmeyi mümkün kılar. **Tabu Search**, yakın zamanda ziyaret edilmiş durumların kısa bir listesini tutarak aynı noktalara dönmeyi önlemeye çalışır. **Simulated Annealing** ise kimi zaman daha kötü hamlelere de kontrollü izin vererek sıkışmadan çıkabilir.



Ek Grafik C. S_2, S_3 ve S_4 aynı sayıda ihlal içeren plato durumlarıdır. Eşit maliyetli hareketler S_5'teki çözüme yol açabilir.

Kısıt ağırlıklandırma (Constraint Weighting)

Kısıt ağırlıklandırmada her kısıt için başlangıç ağırlığı $W_i=1$ tanımlanır. Algoritma ihlal edilen kısıtların **toplam ağırlığını** en aza indirecek atama değişikliğini arar. Ardından mevcut durumda ihlal edilen kısıtların ağırlığı artırılır. Bu işlem (1) plato yüzeyini daha ayırt edilebilir hâle getirebilir; (2) kalıcı olarak sorun oluşturan kısıtlara daha fazla ağırlık yükler.

$$\text{Ağırlıklı ihlal maliyeti} = \Sigma\, W_i \times I_i$$
$$I_i = 1 \text{ (ihlal)}, 0 \text{ (sağlanıyor)}$$

Örneğin C_1'in ağırlığı 5, C_2'nin 1, C_3'ün 3 olsun. Sadece C_1'i ihlal etmek maliyet 5; C_2 ve C_3'ü ihlal etmek maliyet 4 oluşturur. İkinci durumda daha çok sayıda kısıt ihlal edilse de ağırlıklı maliyet küçüktür. Bu, algoritmanın zor kısıtları özel olarak ele almasını sağlar.

Değişen problemleri çevrim içi düzeltme

Yerel aramanın bir başka avantajı, şartları değişen problemlerde mevcut çözümü yeniden kullanabilmesidir. Binlerce uçuş ve on binlerce çalışan atamasına sahip haftalık uçuş programı, tek havalimanındaki kötü hava nedeniyle geçersiz olabilir. Böyle durumda sıfırdan farklı bir program kurmak yerine mevcut çizelgeden başlayıp mümkün olduğunca az sayıda atamayı değiştirerek onarmak uygun olabilir. Yeni kısıtlarla baştan Backtracking yapmak daha uzun sürebilir ve önceki programa göre çok daha fazla değişiklik içeren çözüm üretebilir.

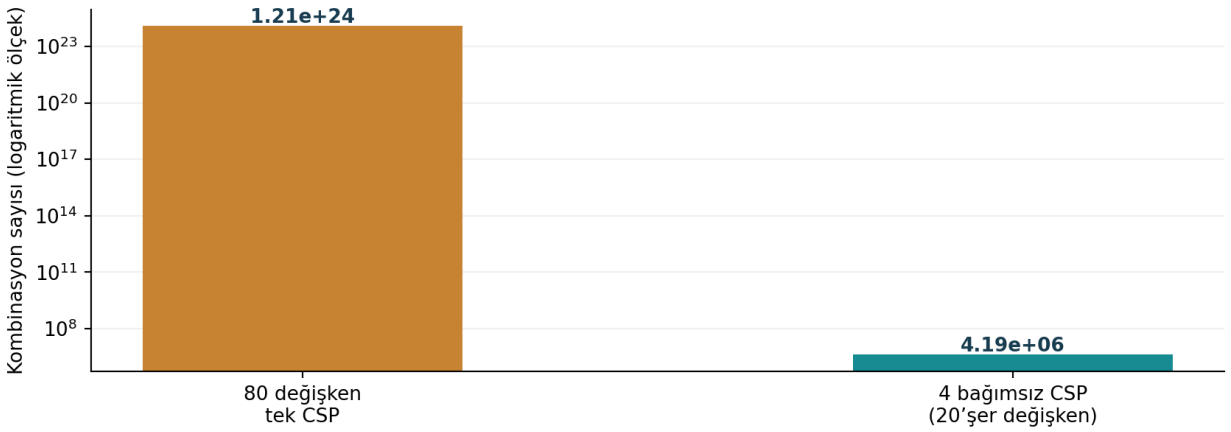
6.5 Problemlerin Yapısı (The Structure of Problems)

Bu bölüm, kısıt grafiğinin yapısının çözümleri hızla bulmak için nasıl kullanılabileceğini inceler. Aynı yaklaşımın olasılıksal akıl yürütme gibi başka alanlarda da uygulamaları vardır. Karmaşık gerçek dünya problemlerini çözebilmek için onları daha küçük alt problemlere ayırmak çoğu zaman temel adımdır.

Bağımsız alt problemler (Independent Subproblems)

Avustralya haritasında Tazmanya anakaraya bağlı değildir. Tazmanya'yı boyama problemi, anakarayı boyamadan bağımsızdır: anakaraya ait herhangi bir çözüm Tazmanya'nın herhangi geçerli rengiyle birleştirilebilir. Kısıt grafiğinin **bağlı bileşenlerini (connected components)** bularak bu ayrışmayı tanıyabiliriz. Her bileşen bağımsız bir CSP_i'dir ve alt çözümlerin birleşimi bütün problemin çözümüdür.

Toplam n değişken, d değer ve her biri sabit c değişken içeren n/c bağımsız alt problem varsa tek tek çözüm yükü en fazla d^c olur. Toplam yük **$O((n/c)d^c)$**'dir; c sabitken n ile doğrusal büyür. Ayrışma olmazsa naif arama $O(d^n)$ düzeyinde üstel olabilir.



Ek Grafik D. 80 Boolean değişkenin bir bütün olarak 2^{80} kombinasyonu ile dört bağımsız 20 değişkenli parçanın 4×2^{20} toplam kombinasyonu arasındaki fark (logaritmik eksen). Bu bir kaba kombinasyon sayımıdır, garantili çalışma süresi değil.

Örneğin n=80, d=2 ve c=20 ise ayrıştırılmadan $2^{80}\approx1{,}21\times10^{24}$ tam atama; dört bağımsız alt problemde toplam $4\times2^{20}=4.194.304$ kombinasyon bulunur. Kitap bu farkı, evrenin ömrüne kıyasla bir saniyeden kısa çözüme geçiş benzetmesiyle vurgular. Gerçek çalışma süresi elbette donanıma ve kullanılan algoritmaya bağlıdır.

Ağaç yapılı CSP'ler (Tree-Structured CSPs)

Bağımsız alt problemler çok kullanışlı olsa da nadirdir. Neyse ki döngüsüz bağlı kısıt grafikleri, yani **ağaçlar**, verimli çözülebilir. Ağaçta herhangi iki değişken arasında tek bir basit yol bulunur. Ağaç yapılı CSP'ler için yeni bir kavram kullanılır: **Directed Arc Consistency (DAC — Yönlü Yay Tutarlılığı)**.

$X_1,...,X_n$ sıralaması için, her X_i kendisinden sonra gelen ve kısıt ilişkisi bulunan X_j'lere (j>i) göre yay-tutarlıysa ağ bu sıraya göre yönlü yay-tutarlıdır. Önce ağacın herhangi bir düğümü kök seçilir; ardından her ebeveynin çocuklarından önce bulunduğu **topolojik sıralama** oluşturulur.

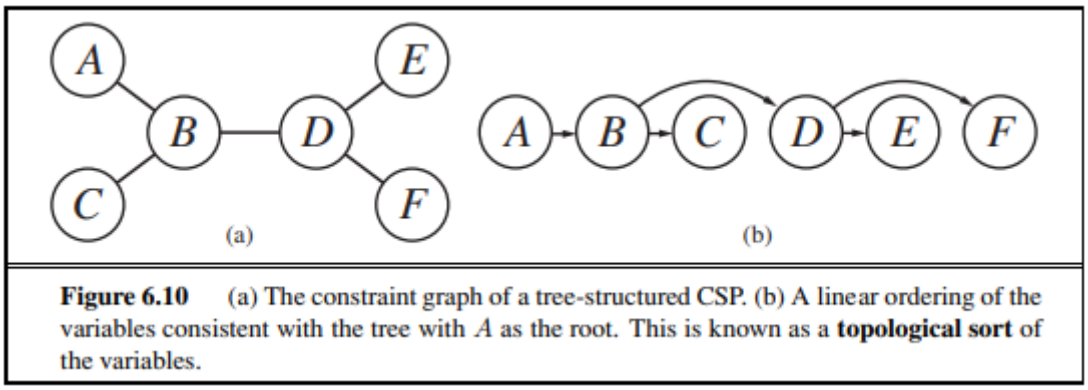


Figure 6.10 (a) The constraint graph of a tree-structured CSP. (b) A linear ordering of the variables consistent with the tree with *A* as the root. This is known as a **topological sort** of the variables.

Şekil 6.10. (a) Ağaç yapısındaki CSP'nin kısıt grafiği. (b) A kök seçilerek hazırlanmış, ağaçla uyumlu A,B,C,D,E,F sıralaması (topolojik sıralama).

Şekil 6.10'da bağlantılar A–B, B–C, B–D, D–E, D–F'dir. Bu grafikte her iki düğüm arasında tek yol vardır ve n=6 için n−1=5 kenar bulunur. Topolojik sıralama A,B,C,D,E,F olabilir; A,B,D,F,E,C de ebeveyn-çocuk sırasını bozmadığı için uygundur.

Ağaçta n−1 kenar bulunduğundan her ebeveyn-çocuk yayı için en fazla $O(d^2)$ kontrol yaparak yönlü tutarlılık **$O(nd^2)$** zamanda sağlanabilir. Ardından kökten yapraklara doğru mevcut değer kümelerinden tutarlı değerler seçilerek **geri izlemesiz** çözüm bulunur. Sabit d için karmaşıklık n'ye göre doğrusaldır.

Şekil 6.11: TREE-CSP-SOLVER algoritması

```
TREE-CSP-SOLVER(csp):
    n ← değişken sayısı
    assignment ← {}
    root ← herhangi bir değişken
    X ← TOPOLOGICAL-SORT(X, root)
    j = n, n-1, ..., 2 için:
        MAKE-ARC-CONSISTENT(PARENT(Xj), Xj)
        tutarlılık sağlanamazsa: başarısız dön
    i = 1, 2, ..., n için:
        assignment[Xi] ← Di içinden tutarlı değer
        değer bulunamazsa: başarısız dön
    assignment döndür
```

Birinci geçişte yapraklardan köke doğru alanlar daraltılır; ikinci geçişte kökten yapraklara doğru değerler seçilir. Yayların önceki tutarlılık kontrolü her ebeveyn değeri için çocukta en az bir destek olmasını sağlar. Böylece arama sırasında geri izleme gerekmez.

> **Çözümlü örnek:** A–B–C doğrusu üzerinde A<B, B<C ve bütün değer kümeleri {1,2,3} olsun. B<C nedeniyle B=3 elenir; B={1,2}. A<B nedeniyle A yalnızca 1 olabilir. İleri atamada A=1, B=2, C=3 seçilerek sonuç bulunur.

Döngü kesme kümesi ve Cutset Conditioning

Ağaç algoritması verimli olduğuna göre daha genel grafikler ağaç biçimine dönüştürülebilir mi? İlk strateji, bazı değişkenlere değer atayıp grafikten çıkararak kalan grafiğin döngüsüz olmasını sağlamaktır. Şekil 6.12(a) Avustralya'nın özgün kısıt grafiğini gösterir. SA değişkeni çıkarıldığında Şekil 6.12(b)'de bir yol (WA–NT–Q–NSW–V) ve bağımsız T düğümü kalır. Teknik olarak bu bir **orman (forest)**dır; her bağlı bileşen ağaçtır ve ayrı çözülür.

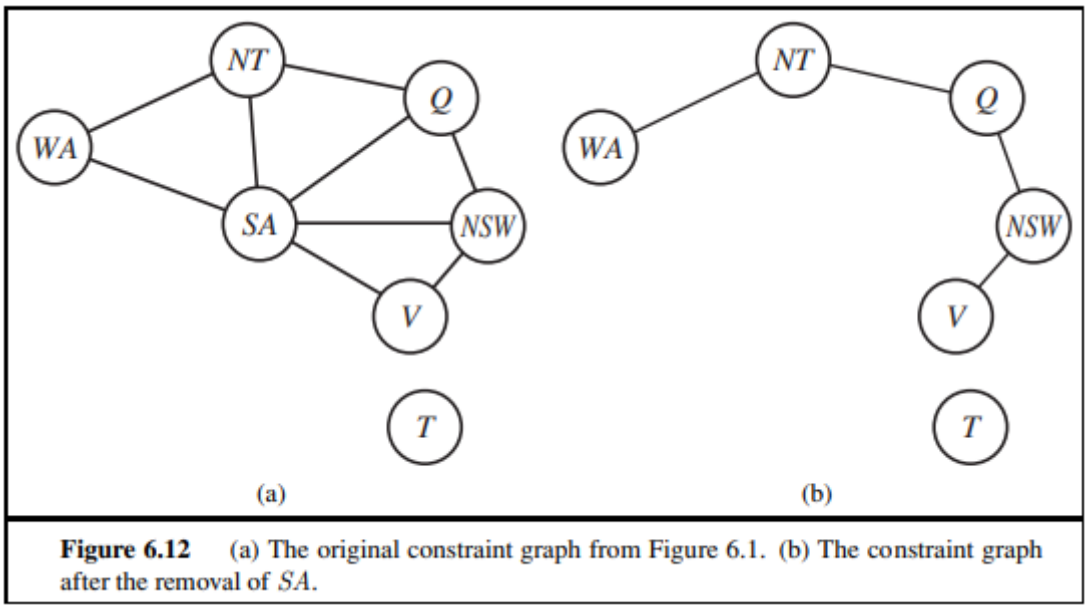


Figure 6.12 (a) The original constraint graph from Figure 6.1. (b) The constraint graph after the removal of *SA*.

Şekil 6.12. (a) Şekil 6.1'deki özgün kısıt grafiği. (b) SA düğümü çıkarıldıktan sonraki döngüsüz grafik.

Grafikteki SA'yı çıkarabilmek için önce SA'ya bir değer sabitlenir ve diğer değişkenlerin değer kümelerinden bu seçimle çelişen değerler silinir. Sonra geriye kalan döngüsüz problem ağaç algoritmasıyla çözülür. SA=mavi seçildiğinde beş komşusundan mavi elenir; kalan yol üzerinde renkleri sırayla değiştirmek ve T'yi bağımsız boyamak mümkündür. Genel problemde ilk SA rengi işe yaramayabilir; bu nedenle SA'nın başka değerleri denenir.

Genel biçimde, çıkarıldığında kalan grafiği döngüsüz bırakan değişkenler kümesi **S** seçilir. Bu kümeye **cycle cutset (döngü kesme kümesi)** denir. S üzerindeki iç kısıtları sağlayan her atama için (1) kalan değer kümelerinden S'yle tutarsız değerler silinir; (2) kalan orman/ağaç çözülür; çözüm bulunursa S ile birleştirilip döndürülür.

Cutset Conditioning: $O(d^c \cdot (n-c)\, d^2)$ $(c = |S|)$

S üzerinde en fazla d^c kombinasyon vardır. Her kombinasyonda $n-c$ değişkenli ağaç çözme $O((n-c)d^2)$ gerektirir. Grafik “neredeyse ağaç”sa c küçüktür ve doğrudan Backtracking'e göre büyük kazanç elde edilir. En kötü durumda c, $n-2$ olabilir. En küçük cutset'i bulmak NP-hard'dır; kullanışlı yaklaşım algoritmaları geliştirilmiştir. Aynı strateji, 14. bölümde olasılıksal akıl yürütmede yeniden görülür.

Sayısal gösterim: Avustralya'da n=7, d=3, S={SA}, c=1 için formülün içindeki çarpım $3^1 \times (7-1) \times 3^2 = 3 \times 6 \times 9 = 162$'dir. Bu gerçek işlem sayısı değil, asimptotik bağımlılığı somutlaştıran bir yerine koymadır.

Ağaç ayrıştırması (Tree Decomposition)

İkinci strateji, kısıt grafiğini birbiriyle örtüşebilen bağlantılı alt problemlere ayırmak ve bu alt problemlerin bağlantılarını ağaç biçiminde düzenlemektir. Her alt problem ayrı çözülür; ardından çözümler ortak değişkenlerin tutarlı olması şartıyla birleştirilir. Böl-ve-yönet yaklaşımında olduğu gibi alt problemlerin küçük tutulması önemlidir.

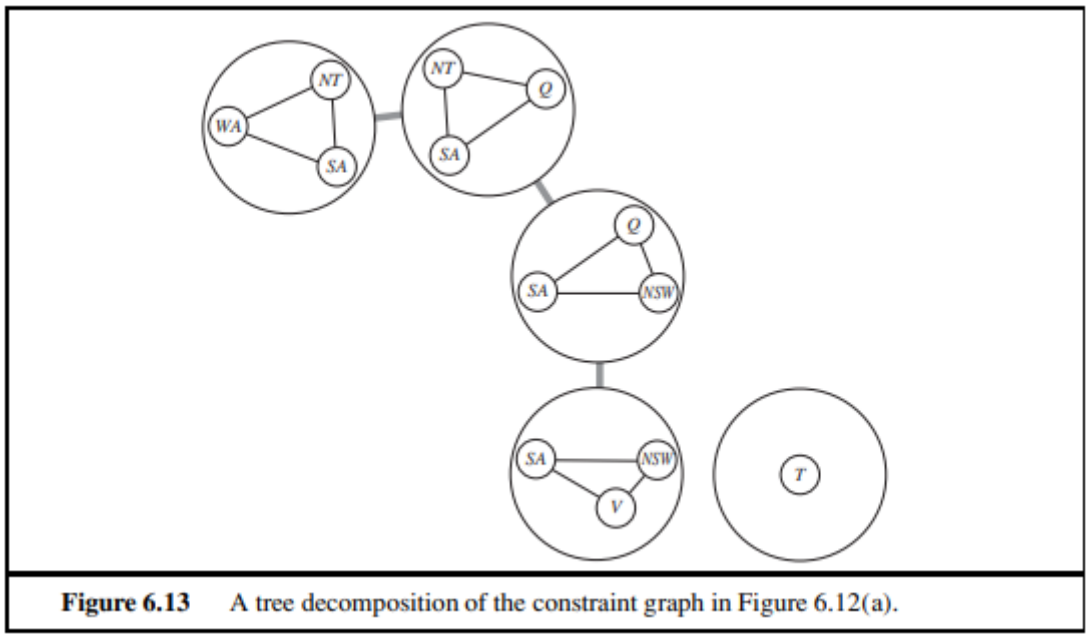


Figure 6.13 A tree decomposition of the constraint graph in Figure 6.12(a).

Şekil 6.13. Şekil 6.12(a)'nın kısıt grafiği için ağaç ayrıştırması. İlk dört alt problem birbirine bağlanır, T ayrı bileşendir. Büyük çemberler alt problem kümelerini gösterir.

Şekilde beş alt problem vardır: {WA,NT,SA}; {NT,SA,Q}; {SA,Q,NSW}; {SA,NSW,V}; {T}. SA, ilk dört alt problemde tekrar görünür; bütün görünümlerinde aynı renk değeri kullanılmalıdır.

Ağaç ayrıştırması için üç koşul

1. Orijinal problemdeki **her değişken** en az bir alt problemde yer almalıdır.
2. Bir kısıtla birbirine bağlı iki değişken, kısıtlarıyla beraber **en az bir alt problemde** birlikte yer almalıdır.
3. Bir değişken iki alt problemde bulunuyorsa, bu alt problemleri bağlayan ağaç yolu üzerindeki **her alt problemde** de bulunmalıdır (running-intersection koşulu).

İlk iki koşul değişken/kısıt kapsamını, üçüncü koşul tekrar edilen değişkenlerin alt problemler boyunca tutarlı değer almasını güvence altına alır. Alt problemler önce yerel olarak çözülür; birinde çözüm yoksa bütün CSP çözümsüzdür. Tümünde çözüm varsa her alt problem, değer kümesi kendi yerel çözümlerinden oluşan **mega-değişken** gibi ele alınır. Alt problemleri birbirine bağlayan kısıtlar yalnızca ortak değişkenlerde eşit atama gerektirir. Böylece ağaç algoritmasıyla birleştirme yapılır.

Şekil 6.13'ün sayılarla çözümü

Birinci alt problem üçgen olduğundan {WA,NT,SA} için üç farklı renk atanmalıdır; 3!=6 geçerli yerel çözüm vardır. Örneğin {WA=kırmızı, SA=mavi, NT=yeşil} seçilsin. İkinci alt problem {NT,SA,Q} bu ortak değerleri korumak zorundadır; bu nedenle Q=kırmızı tek seçenektir. Üçüncü alt problem {SA,Q,NSW} için NSW=yeşil olur. Dördüncü alt problem {SA,NSW,V} için V=kırmızı olur. Son alt problem T bağımsızdır; T=kırmızı seçilebilir. Bütün değişkenleri kapsayan tutarlı çözüm elde edilir.

Ağaç genişliği (Tree Width)

Aynı grafik birçok farklı ağaç ayrıştırmasına sahip olabilir; amaç en büyük alt problemi mümkün olduğunca küçük tutmaktır. Bir ayrıştırmanın **tree width** değeri, en büyük alt problemdeki değişken sayısından **bir eksiktir**. Grafiğin tree width değeri, mümkün bütün ayrıştırmalardaki en küçük bu değerdir.

Bir grafiğin tree width değeri w ve bu genişliğe sahip ayrıştırması verilmişse CSP, **$O(nd^{w+1})$** zamanda çözülebilir. Dolayısıyla sınırlı tree width değerine sahip CSP aileleri polinom zamanda çözülebilir. Ne yazık ki en küçük genişlikli ayrıştırmayı bulmak NP-hard'dır; pratikte sezgisel yöntemler kullanılır.

> **Şekil 6.13 için:** En büyük alt problem 3 değişken içerir, dolayısıyla gösterilen ayrıştırmanın genişliği $3-1=2$'dir. Grafikte üçlü döngüler bulunduğundan minimum genişlik de 2'dir. n=7, d=3, w=2 için büyüklük ifadesi $7\times3^3=189$ olur (tam işlem sayısı değildir).

Değer simetrisi ve simetri kırma (Value Symmetry)

Problem yapısı yalnızca grafikte değil, değişkenlerin değerlerinde de bulunabilir. n renkli harita boyama problemlerinde renk adlarını yer değiştirerek aynı yapısal çözümün n! farklı adlandırması üretilebilir. WA, NT ve SA'nın üç farklı renkte olması gerekir; üç rengi bu üç bölgeye atamanın 3!=6 farklı yolu vardır. Buna **değer simetrisi** denir.

Arama uzayını bu eşdeğer atamalar açısından küçültmek için **symmetry-breaking constraint (simetri kırma kısıtı)** eklenebilir. Örneğin renk adlarını alfabetik olarak blue < green < red biçiminde sıralayıp NT < SA < WA kısıtı koyarsak altı simetrik çözümden yalnızca NT=mavi, SA=yeşil, WA=kırmızı kalır. Bu tür simetri kırmanın amacı, gerçekten farklı çözüm sınıflarını kaybetmeden tekrarlanan adlandırmaları elemektir.

Harita boyama örneğinde simetri kırmak kolaydır. Genel CSP'de tam çözümlerin simetrik eşdeğerlerini ayıklamak için polinom süreli kısıtlar bulunabilse de arama sırasındaki bütün *ara* simetrileri kaldırmak NP-hard olabilir. Yine de pratik çözücülerde değer simetrisini kırmak geniş bir problem yelpazesinde yararlı bulunmuştur.

Bölüm Sonu: Genel Tekrar ve Çözümlü Alıştırmalar

Altı ana yaklaşımın ilişkisi

Bir problemi CSP olarak modellemek, çözüm algoritmasını belirlemekten farklıdır. CSP=(X,D,C) yalnızca değişkenleri, alanları ve kuralları anlatır. Algoritma seçiminde problemin tipi ve yapısı önemlidir:

Durum	Önerilen yaklaşım	Temel fikir
Tekli kısıtlar çok	Node consistency	Uygunsuz değerleri başta sil
İkili kısıtlar ve dar alanlar	AC-3 / MAC	Alanları yinelemeli daralt
Arama gerekli, kurallar katı	Backtracking + MRV + LCV	Akıllı seçim, erken budama
Tam bir aday çözüm kolay kuruluyor	Min-Conflicts	Çakışmaları azaltarak onar
Grafik bağımsız parçalara ayrılıyor	Connected components	Alt problemleri ayrı çöz
Grafik ağaç / ağaca yakın	Tree solver / Cutset / Tree decomposition	Yapıdan yararlan
Birçok eşdeğer atama var	Symmetry breaking	Aynı çözümü tekrar arama

Öğrencinin bilmesi gereken formüller

Formül	Ne ifade eder?
CSP=(X,D,C)	Değişken, değer kümesi, kısıt
$n! \cdot d^n \rightarrow d^n$	Değişme özelliğiyle tekrar atama sıralarını kaldırma
$T_1 + d_1 \leq T_2$	İş çizelgelemede öncelik kısıtı
$O(cd^3)$	AC-3 standart en kötü durum zaman sınırı
$O(nd^2)$	Ağaç yapılı CSP çözümü
$O(d^c(n-c)d^2)$	c değişkenli cutset conditioning
$O(nd^{w+1})$	Ağaç genişliği w olan ayrıştırılmış CSP
3!=6	Üç farklı rengi üç bölgeye sıralı atama sayısı

Alıştırma 1 — CSP'yi kur

Soru: A, B ve C dersleri Pazartesi, Salı, Çarşamba günlerinden birine yerleştirilecek. A ve B aynı gün, B ve C aynı gün olamaz; C yalnızca Salı veya Çarşamba olabilir. Değişkenleri, alanları ve kısıtları yaz; bir çözüm bul.

Çözüm: X={A,B,C}; D_a=D_β={Pzt,Sal,Çar}; D_c={Sal,Çar}. Kısıtlar A≠B, B≠C. Örneğin A=Pzt, B=Sal, C=Çar seçimi geçerlidir. A ve C'nin aynı gün olmasını yasaklayan bir kısıt olmadığına dikkat et.

Alıştırma 2 — Atama türünü belirle

Soru: A≠B, B≠C kısıtları bulunan üç değişkenli CSP'de A=1, B=2; A=1,B=1,C=3; A=1,B=2,C=2 atamalarını değerlendir.

Çözüm: İlk atama kısmi ama tutarlıdır. İkinci atama tam fakat A=B olduğu için tutarsızdır. Üçüncü tam fakat B=C olduğu için tutarsızdır. Hiçbiri çözüm değildir. A=1,B=2,C=3 hem tam hem tutarlı bir çözüm olabilir.

Alıştırma 3 — Yay tutarlılığı

Soru: X,Y∈{1,2,3,4}, Y=2X olsun. Her iki değer kümesini yay-tutarlı hâle getir.

Çözüm: İzin verilen çiftler (1,2),(2,4)'tür; X=3,4 için Y=6,8 alan dışında kalır. Y=1 ve Y=3 hiçbir X tarafından desteklenmez. Böylece D_x={1,2}, D_Y={2,4}.

Alıştırma 4 — AC-3 çıkarımı

Soru: A,B,C∈{1,2,3}, A<B<C. AC-3'ün alanları nasıl daralttığını adım adım göster.

Çözüm: A<B yüzünden A=3 elenir. B<C yüzünden B=3 elenir. B'nin 1 değeri A'da kendisinden küçük bir değer olmadığından elenir. Böylece B=2 olur. A<B → A=1; B<C → C=3. Sonuç (1,2,3).

Alıştırma 5 — Yol tutarlılığı neden gerekli?

Soru: X,Y,Z üçgeni için X≠Y, Y≠Z, X≠Z ve hepsi {kırmızı,mavi} alanına sahip olsun. Ağ yay-tutarlı mıdır? Çözüm var mı?

Çözüm: Her ikili kenarda seçilen renge karşılık komşuda farklı renk vardır; yay-tutarlıdır. Fakat üçlü tamamen birbirine komşu olduğundan iki renk yetmez, çözüm yoktur. Yol tutarlılığı iki değişkene farklı renk verilince üçüncüye hiçbir renk kalmadığını saptar.

Alıştırma 6 — MRV, Degree, LCV

Soru: Atanmamış A={1,2,3}, B={2}, C={1,2} ise MRV hangi değişkeni seçer? B atandıktan sonra A ve C'nin iki seçeneği kalırsa ve C üç atanmamış değişkenle, A yalnızca bir değişkenle ilişkiliyse Degree hangisini seçer?

Çözüm: MRV B'yi seçer; çünkü bir değeri vardır. Sonraki eşitlikte Degree, daha çok komşu kısıtı bulunan C'yi seçer. LCV ise seçilen değişkenin *değerlerini* komşularda en az alan daraltması yapacak şekilde sıralar.

Alıştırma 7 — Forward Checking ve MAC

Soru: X,Y,Z∈{1,2}; X≠Y, Y≠Z, X≠Z. X=1 atandıktan sonra Forward Checking ve MAC arasındaki farkı göster.

Çözüm: Forward Checking, Y ve Z'den 1'i siler; Y=Z=2 kalır, ancak atanmamış Y ile Z arasındaki kısıtı bu aşamada yeniden kontrol etmez. MAC yayılımı Y≠Z'yi de işlediğinde aynı değerin iki değişkende zorunlu olduğunu fark eder, alanlardan biri boşalır ve çelişkiyi hemen bildirir.

Alıştırma 8 — Min-Conflicts

Soru: A,B,C∈{1,2,3}, hepsi farklı olmalı. Başlangıç A=2,B=2,C=1. B'nin her olası değeri için çakışma sayısını hesapla.

Çözüm: B=1 → C ile 1 çakışma; B=2 → A ile 1; B=3 → 0. Dolayısıyla B=3 seçilir; sonuç {2,3,1} çözümdür.

Alıştırma 9 — Tree width

Soru: Bir ağaç ayrıştırmasının alt problem boyutları {2,3,4,2,1} olsun. Gösterilen ayrıştırmanın ağaç genişliği kaçtır? n=20, d=3 için $O(nd^{w+1})$ ifadesinin örnek büyüklüğü nedir?

Çözüm: En büyük alt problem 4 değişken içerir; w=4−1=3. $n \cdot d^{w+1} = 20 \times 3^4 = 20 \times 81 = 1620$ olur. Bu çarpım tam işlem sayısı değildir; asimptotik büyümeyi örnekler.

Alıştırma 10 — Simetri kırma

Soru: Üç komşu bölge üç farklı renkle boyanacak. Renk isimleri çözümlerin anlamını değiştirmiyorsa kaç eşdeğer atama vardır? Bu atamaları azaltmak için ne yaparsın?

Çözüm: 3!=6 farklı adlandırma vardır. Bölgeler arasında sabit ve keyfî bir renk sırası belirleyen simetri kırma kısıtı eklenebilir; böylece eşdeğer sınıftan tek temsilci kalır. Bu yöntem, renklerin özel bir anlamı olmadığı varsayımına dayanır.

Terimler sözlüğü

İngilizce	Türkçe
Constraint Satisfaction Problem	Kısıt Tatmin Problemi
Variable / Domain / Constraint	Değişken / Değer kümesi / Kısıt
Assignment / Consistent / Complete	Atama / Tutarlı / Tam
Constraint Propagation	Kısıt yayılımı
Node / Arc / Path Consistency	Düğüm / Yay / Yol tutarlılığı
Generalized Arc Consistency	Genelleştirilmiş yay tutarlılığı
Backtracking / Backjumping	Geri izleme / Geri sıçrama
Conflict Set / No-Good	Çakışma kümesi / Geçersiz atama kombinasyonu
Forward Checking / MAC	İleri kontrol / Yay tutarlılığını koruma
Min-Conflicts	En az çakışma
Cycle Cutset / Tree Decomposition	Döngü kesme kümesi / Ağaç ayrıştırması
Tree Width / Symmetry Breaking	Ağaç genişliği / Simetri kırma

Kapanış: CSP'yi tek cümlede hatırla

CSP, bir problemin değişkenlerini, izin verilen değerlerini ve kısıtlarını açıkça tanımlayarak; çıkarım, arama ve yapısal ayrıştırma yardımıyla tutarlı bir tam atama bulma yaklaşımıdır. Problemin bütün ayrıntılarını sıfırdan algoritmaya yazmak yerine doğru modeli kurmak, genel amaçlı çözücülerin gücünden yararlanmayı sağlar.

Kaynak ve kapsam notu: Russell, S. J. & Norvig, P. (2010). *Artificial Intelligence: A Modern Approach* (3rd ed.), Chapter 6. Bu dosya, kullanıcı tarafından sağlanan 6.1–6.5 pasajlarının bölüm sırasını temel alan Türkçe çeviri ve genişletilmiş pedagojik açıklamalar içerir. Şekil 6.1, 6.2, 6.4, 6.6, 6.7, 6.9, 6.10, 6.12 ve 6.13 kullanıcı tarafından paylaşılan görsellerden alınmıştır; diğer çizimler bu not için hazırlanmıştır. Kitabın ayrı alıştırma seti ve kullanıcı tarafından iletilmeyen başka kısımlar bu belgenin kaynak metnine dahil değildir.